

تطوير نظام إدارة المصروفات باستخدام PHP وMySQL

فهرس الدروس

١. مقدمة المشروع والتعريف به.

٢. تجهيز بيئة العمل والأدوات المطلوبة.

٣. إنشاء قاعدة البيانات والجداول.

٤. هيكل المشروع والملفات العامة.

٥. ملف الاتصال بقاعدة البيانات.

٦. التسجيل وإنشاء حساب جديد.

٧. تسجيل الدخول والخروج.

٨. الصفحة الرئيسية

٩. إنشاء صفحة التصنيفات.

١٠. إضافة تصنيف جديد.

١١. تعديل التصنيف.

١٢. حذف التصنيف.

١٣. صفحة المصروفات.

١٤. إضافة مصروف جديد.

١٥. تعديل المصروف.

١٦. حذف المصروف.

١٧. صفحة الميزانية

١٨. لوحة التحكم الرئيسية.

١٩. تعديل الملف الشخصي

الدرس الأول: مقدمة المشروع والتعريف به

1) ما هو مشروع إدارة المصروفات؟

مشروع إدارة المصروفات هو تطبيق ويب يساعد المستخدم على تسجيل ما يصرفه يوميًا أو شهريًا، ثم يعرض له هذه البيانات بطريقة منظمة حتى يعرف أين يذهب المال. الفكرة بسيطة، لكنها مهمة جدًا لأنها تحول الأرقام العشوائية إلى معلومات واضحة يمكن الاعتماد عليها.

2) لماذا اخترنا هذا المشروع؟

اخترنا هذا المشروع لأنه:

- عملي ومفيد في الحياة اليومية.
- يحتوي على أكثر من جزء تعليمي.
- يوضح التعامل مع قاعدة البيانات.
- يشرح تنظيم البيانات داخل جداول.
- يسمح بإضافة ميزات متقدمة مثل التقارير والرسوم البيانية والتنبيهات.

3) فكرة المشروع باختصار

الفكرة الأساسية هي أن يقوم المستخدم بإنشاء حساب، ثم يسجل دخوله، وبعدها يستطيع:

- إضافة المصروفات.
- تصنيف المصروفات.
- مشاهدة التقارير.
- تحديد ميزانية شهرية.
- متابعة نسبة الصرف.
- الحصول على تنبيهات عند الاقتراب من الحد.

بهذا الشكل يصبح النظام ليس مجرد سجل للمصروفات، بل أداة تساعد المستخدم على التحكم في مصاريفه واتخاذ قرارات مالية أفضل.

4) ما الذي سيتضمنه المشروع النهائي؟

المشروع النهائي الذي سنبنيه سيتكون من عدة أجزاء مترابطة، وأهمها:

- صفحة التسجيل.
- صفحة تسجيل الدخول.
- الصفحة الرئيسية أ

- لوحة التحكم.
- إدارة التصنيفات.
- إدارة المصروفات.
- صفحة الملف الشخصي.
- تسجيل الخروج.

هذا التقسيم يجعل المشروع منظماً وسهل الفهم، خصوصاً للمبتدئين الذين يتعلمون لأول مرة كيف تُبنى المشاريع الكاملة من البداية إلى النهاية.

5) ماذا سيتعلم الطالب من هذا المشروع؟

من خلال هذا المشروع سيتعلم الطالب عدة مهارات مهمة، منها:

- إنشاء قاعدة بيانات وعلاقات بين الجداول.
- استخدام PHP لمعالجة البيانات.
- الربط بين PHP و MySQL.
- التعامل مع النماذج Forms.
- استخدام الجلسات Sessions.
- تنفيذ عمليات الإضافة والتعديل والحذف.
- عرض البيانات داخل جداول.
- بناء تقارير وتصفية البيانات.
- إضافة رسوم بيانية وتحسينات للواجهة.

6) ما هي مميزات المشروع؟

هذا المشروع يتميز بعدة نقاط مهمة:

- بسيط من حيث الفكرة.
- غني من حيث المحتوى التعليمي.
- قابل للتطوير.
- مناسب لشرح القواعد الأساسية في قواعد البيانات.
- مناسب للمبتدئين لأنه لا يبدأ بميزات معقدة جداً.

7) ما هي التقنية المستخدمة؟

في هذا المشروع سنستخدم:

- **PHP** لكتابة منطق الموقع.
 - **MySQL** لتخزين البيانات.
 - **HTML** لبناء الصفحات.
 - **CSS** لتنسيق الشكل.
 - **JavaScript** لبعض التفاعلات.
- استخدام هذه التقنيات معًا يعطي مشروعًا كاملاً ومناسبًا جدًا للتطبيق العملي.

الدرس الثاني: تجهيز بيئة العمل والأدوات المطلوبة

مقدمة الدرس

قبل أن نبدأ في كتابة أي كود أو إنشاء أي جدول، يجب أن نجهز بيئة العمل بشكل صحيح. هذه الخطوة مهمة جدًا لأنها تجعل تنفيذ المشروع أسهل، وتمنع ظهور أخطاء كثيرة أثناء التطبيق. في هذا الدرس سنتعرف على الأدوات التي نحتاجها، وكيف نجهز الجهاز حتى نبدأ المشروع بشكل عملي ومنظم.

هدف الدرس

هدف هذا الدرس هو أن يعرف الطالب:

- ما الأدوات اللازمة لتنفيذ مشروع إدارة المصروفات.
- كيف يشغل PHP و MySQL على جهازه.
- كيف يجهز مكان المشروع داخل الجهاز.
- كيف يفتح المشروع ويبدأ العمل عليه خطوة بخطوة.

الأدوات المطلوبة

حتى نبدأ المشروع نحتاج إلى الأدوات التالية:

- **XAMPP** أو أي برنامج مشابه مثل WAMP.
- محرر أكواد مثل Visual Studio Code.
- متصفح مثل Google Chrome.
- قاعدة بيانات MySQL.
- **phpMyAdmin** لإدارة قاعدة البيانات بسهولة.
- مجلد خاص بالمشروع داخل مجلد التشغيل المحلي.

لماذا نستخدم XAMPP ؟

XAMPP هو أحد أفضل البرامج للمبتدئين لأنه يجمع أكثر من خدمة في مكان واحد. فهو يوفر لنا:

- خادم Apache لتشغيل ملفات PHP.
- قاعدة بيانات MySQL لتخزين البيانات.
- **phpMyAdmin** لإدارة الجداول والبيانات بطريقة سهلة.

هذا يعني أننا لا نحتاج إلى تثبيت كل جزء بشكل منفصل، بل نحصل على بيئة متكاملة تساعدنا على تشغيل المشروع محليًا على الجهاز.

خطوات تثبيت وتجهيز البيئة

الآن سنشرح الخطوات بشكل بسيط وواضح:

1) تحميل XAMPP

أول خطوة هي تحميل برنامج XAMPP من الموقع الرسمي ثم تثبيته على الجهاز. أثناء التثبيت يفضل ترك الإعدادات الافتراضية كما هي.

2) تشغيل الخدمات الأساسية

بعد التثبيت، نفتح لوحة تحكم XAMPP ثم نشغل:

• Apache

• MySQL

عند تشغيل هاتين الخدمتين يصبح الجهاز جاهزاً لتشغيل صفحات PHP وربطها بقاعدة البيانات.

3) التأكد من عمل السيرفر

يمكن فتح المتصفح وكتابة:

• localhost

إذا ظهرت صفحة XAMPP أو صفحة الاستقبال، فهذا يعني أن الخادم يعمل بشكل صحيح.

4) إنشاء مجلد المشروع

نذهب إلى مجلد htdocs داخل XAMPP ، ثم ننشئ مجلدًا جديدًا باسم المشروع، مثل:

• expense_tracker

هذا المجلد سيكون مكان جميع ملفات المشروع.

5) فتح المشروع في محرر الأكواد

بعد إنشاء المجلد، نفتح Visual Studio Code أو أي محرر آخر، ثم نفتح المجلد الخاص بالمشروع حتى نبدأ بكتابة الملفات داخله.

6) كيف ننظم ملفات المشروع؟

حتى يكون المشروع مرتبًا من البداية، من الأفضل أن نقسمه إلى ملفات واضحة. مثلًا:

• ملف الاتصال بقاعدة البيانات.

• ملفات الهيدر والفوتر.

• ملفات CSS و JavaScript.

• صفحات التصنيفات.

• صفحات المصروفات.

• صفحات الميزانية.

- صفحات تسجيل الدخول والتسجيل.
 - صفحة الملف الشخصي
- هذا التنظيم يسهل علينا التعديل لاحقًا ويجعل المشروع أكثر احترافية.

ما الذي سنفعله بعد هذا الدرس؟

بعد تجهيز البيئة بنجاح، ننتقل مباشرة إلى الخطوة التالية وهي إنشاء قاعدة البيانات والجداول. هذه ستكون الأساس الذي يعتمد عليه المشروع كله، ولذلك سنشرحها بتفصيل في الدرس الثالث.

خلاصة الدرس

في هذا الدرس تعلمنا كيف نجهز بيئة العمل لمشروع إدارة المصروفات. عرفنا الأدوات المطلوبة، وكيف نشغل الخادم المحلي، وكيف ننشئ مجلد المشروع، ولماذا هذه الخطوة مهمة قبل البدء في البرمجة. هذه المرحلة هي الأساس الذي سنبنى عليه باقي الدروس، لذلك يجب تنفيذها بدقة وهدوء.

الدرس الثالث: إنشاء قاعدة البيانات والجداول

مقدمة الدرس

في هذا الدرس سنبنّي الأساس الحقيقي للمشروع، وهو قاعدة البيانات. قاعدة البيانات هي المكان الذي سنحفظ فيه كل المعلومات الخاصة بالنظام، مثل بيانات المستخدمين، والتصنيفات، والمصروفات، والميزانية، والتنبيهات، والمصروفات المتكررة. إذا كانت قاعدة البيانات مرتبة من البداية، سيكون العمل على باقي أجزاء المشروع أسهل وأوضح بكثير.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- إنشاء قاعدة البيانات.
- تحديد الجداول المطلوبة.
- فهم وظيفة كل جدول.
- كتابة أوامر SQL الخاصة بكل جدول.
- التأكد من أن قاعدة البيانات تدعم اللغة العربية بشكل صحيح.

ما معنى قاعدة البيانات؟

قاعدة البيانات هي مكان منظم لتخزين المعلومات بطريقة تسهّل الوصول إليها وتعديلها وحذفها. في مشروعنا، لا يمكن تخزين المصروفات أو التصنيفات أو الحسابات داخل صفحات PHP فقط، بل يجب حفظها في قاعدة بيانات حتى تبقى محفوظة ويمكن الرجوع إليها في أي وقت.

اسم قاعدة البيانات

يمكننا تسمية قاعدة البيانات بهذا الاسم:

expense_tracker_db

ويفضل أن يكون الاسم واضحًا وسهل الفهم.

دعم اللغة العربية

بما أن المشروع سيستخدم اللغة العربية في كثير من الحقول والنصوص، يجب أن تكون قاعدة البيانات والجداول باستخدام الترميز:

utf8mb4

والتجميع:

utf8mb4_unicode_ci

هذا مهم جدًا حتى يتم حفظ النص العربي بشكل صحيح بدون مشاكل في العرض أو التخزين.

الجدول المطلوبة

سننشئ عدة جداول، وكل جدول له وظيفة محددة داخل المشروع.

1) جدول المستخدمين **users**

هذا الجدول خاص بتخزين بيانات المستخدمين الذين يسجلون في النظام.

وظيفة الجدول

- حفظ اسم المستخدم.
- حفظ البريد الإلكتروني.
- حفظ كلمة المرور.
- حفظ تاريخ إنشاء الحساب.

كود SQL

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  full_name VARCHAR(150) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB  
DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_unicode_ci;
```

شرح بسيط

- id: رقم تلقائي لكل مستخدم.
- full_name: الاسم الكامل للمستخدم.
- email: البريد الإلكتروني، ويجب أن يكون غير مكرر.
- password: كلمة المرور.
- created_at: تاريخ إنشاء الحساب.

2) جدول التصنيفات **categories**

هذا الجدول يخزن أنواع المصروفات مثل الطعام والمواصلات والدراسة والتسوق.

وظيفة الجدول

تنظيم المصروفات داخل تصنيفات واضحة.

كود SQL

```
CREATE TABLE categories (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(150) NOT NULL UNIQUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB  
DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_unicode_ci;
```

شرح بسيط

- id: رقم التصنيف.
- name: اسم التصنيف.
- created_at: تاريخ الإضافة.

3) جدول المصروفات expenses

هذا هو الجدول الأساسي في المشروع، لأنه يخزن كل عملية صرف يقوم بها المستخدم.

وظيفة الجدول

- حفظ المصروفات اليومية.
- ربط كل مصروف بالمستخدم.
- ربط كل مصروف بتصنيف معين.

كود SQL

```
CREATE TABLE expenses (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  category_id INT NOT NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  description VARCHAR(255) NULL,  
  expense_date DATE NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_expenses_user  
    FOREIGN KEY (user_id) REFERENCES users(id)  
    ON DELETE CASCADE,  
  CONSTRAINT fk_expenses_category
```

```
FOREIGN KEY (category_id) REFERENCES categories(id)  
ON DELETE RESTRICT
```

```
) ENGINE=InnoDB
```

```
DEFAULT CHARSET=utf8mb4
```

```
COLLATE=utf8mb4_unicode_ci;
```

شرح بسيط

- user_id: يربط المصروف بالمستخدم.
- category_id: يربط المصروف بالتصنيف.
- amount: قيمة المصروف.
- description: وصف المصروف.
- expense_date: تاريخ المصروف.
- created_at: تاريخ الإضافة.

ملاحظات مهمة

- استخدمنا ON DELETE CASCADE مع المستخدم، حتى إذا تم حذف المستخدم تُحذف بياناته المرتبطة.
- استخدمنا ON DELETE RESTRICT مع التصنيف حتى لا يُحذف تصنيف مرتبط بمصروفات إلا بعد معالجة البيانات.

4) جدول الميزانيات budgets

هذا الجدول يخزن الميزانية الشهرية لكل مستخدم.

وظيفة الجدول

- حفظ ميزانية كل شهر.
- مقارنة المصروفات بالميزانية.
- معرفة نسبة الاستهلاك.

كود SQL

```
CREATE TABLE budgets (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  month TINYINT NOT NULL,  
  year YEAR NOT NULL,  
  budget_amount DECIMAL(10,2) NOT NULL,
```

```

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_budgets_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE,
UNIQUE KEY unique_user_budget (user_id, month, year)
) ENGINE=InnoDB
DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;

```

شرح بسيط

- user_id: يحدد صاحب الميزانية.
- month: الشهر.
- year: السنة.
- budget_amount: قيمة الميزانية.
- unique_user_budget: يمنع تكرار نفس الميزانية لنفس المستخدم في نفس الشهر والسنة.

إنشاء قاعدة البيانات كاملة

يمكن إنشاء قاعدة البيانات أولاً بهذا الكود:

```

CREATE DATABASE expense_tracker_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

```

ثم نختارها:

```
USE expense_tracker_db;
```

بعد ذلك ننفذ أوامر إنشاء الجداول بالترتيب المناسب.

الترتيب المقترح لإنشاء الجداول

الأفضل أن ننشئ الجداول بهذا الترتيب:

١. users

٢. categories

٣. expenses

٤. budgets

هذا الترتيب مهم لأن بعض الجداول تعتمد على جداول أخرى من خلال المفاتيح الخارجية.

خلاصة الدرس

في هذا الدرس أنشأنا الأساس الحقيقي للمشروع، وهو قاعدة البيانات والجداول. تعرفنا على أسماء الجداول بالإنجليزية، ووظيفة كل جدول، وكود SQL الكامل لكل واحد منها. كما تأكدنا من دعم اللغة العربية باستخدام utf8mb4. الآن أصبحت قاعدة البيانات جاهزة، وفي الدرس القادم سنبدأ بربطها مع PHP من خلال ملف الاتصال.

الدرس الرابع : هيكل المشروع والملفات العامة

مقدمة الدرس

بعد أن جهزنا قاعدة البيانات ، نحتاج الآن إلى تنظيم شكل المشروع من الداخل. هذا التنظيم مهم جدًا لأنه يجعل الصفحات متناسقة، ويمنع تكرار الأكواد، ويسهل علينا إدارة المشروع وتطويره لاحقًا.

في هذا الدرس سننشئ الملفات العامة التي سنستخدم في جميع الصفحات، مثل header و footer وملف التنسيق style.css وملف JavaScript، وسنشرح وظيفة كل ملف بطريقة بسيطة وواضحة للمبتدئين.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيف ننظم ملفات المشروع.
- ما وظيفة الملفات العامة.
- كيف نستخدم header و footer في كل الصفحات.
- كيف نجعل المشروع مرتبًا واحترافيًا.
- كيف نبدأ ببناء واجهة موحدة للمستخدم.

لماذا نحتاج إلى هيكل عام؟

إذا كتبنا نفس الشريط العلوي أو نفس التذييل في كل صفحة بشكل مستقل، فسنكرر نفس الكود كثيرًا. وهذا يجعل المشروع:

- أكبر من اللازم.
- أصعب في التعديل.
- أكثر عرضة للأخطاء.

لذلك من الأفضل أن نضع الأجزاء المشتركة في ملفات مستقلة ثم نستخدمها في كل الصفحات. هذا الأسلوب هو الأسلوب الصحيح في المشاريع المنظمة.

الملفات العامة في المشروع

الملفات العامة التي سنعتمد عليها عادة هي:

- header.php
- footer.php
- auth.php
- style.css
- script.js

ملف header.php

ملف header.php هو بداية كل صفحة تقريباً داخل المشروع. فكرته أنه يضم العناصر المشتركة التي نريد أن تظهر في الأعلى بكل الصفحات، مثل الشعار والقائمة الرئيسية.

ماذا يحتوي؟

عادة يحتوي على:

- بداية مستند HTML.
- تحديد اللغة العربية.
- إضافة ملف CSS.
- قائمة التنقل.
- روابط الصفحات الأساسية.

اين يتم انشاء الملف:

في البداية يتم انشاء مجلد **includes**

ثم انشاء ملف header.php داخل هذا المجلد

Header.php

```
<?php
include 'includes/auth.php';
include 'includes/config.php';
?>

<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="theme-color" content="#2563eb">
  <title>نظام إدارة المصروفات</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```



```

<link
href="https://fonts.googleapis.com/css2?family=Cairo:wght@400;600;700;800;900
&display=swap" rel="stylesheet">
<link rel="stylesheet" href="assets/css/style.css">
<script defer src="assets/script/script.js"></script>
</head>
<body>
<header class="site-header">
<div class="container topbar">
<div class="brand">
<a href="dashboard.php" class="brand-mark" aria-label="العودة للرئيسية">
<svg viewBox="0 0 24 24" aria-hidden="true"><path d="M12 2.5 3 7v10l9
4.5 9-4.5 7l-9-4.5zm0 2.24L18.82 8 12 11.27 5.18 8 12 4.74zM5 9.62l6 3v7.86l-6-
3V9.62zm14 0v7.86l-6 3v-7.86l6-3z"/></svg>
</a>
<div class="brand-text">
<h1>Expense Tracker</h1>
<p>إدارة المصروفات</p>
</div>
</div>

<div class="nav-wrap">
<button class="nav-toggle" type="button" data-nav-toggle aria-
expanded="false" aria-label="فتح القائمة">
<span></span>
</button>
<nav class="main-nav" data-nav-menu>

<a href="index.php" class="<?php echo $currentPage === 'index.php' ?
'active' : ''; ?>">الرئيسية</a>
<a href="dashboard.php" class="<?php echo $currentPage ===
'dashboard.php' ? 'active' : ''; ?>">لوحة التحكم</a>
<a href="budget.php" class="<?php echo $currentPage === 'budget.php' ?
'active' : ''; ?>">الميزانية الشهرية</a>

```

```

        <a href="categories.php" class="<?php echo $currentPage ===
'categories.php' ? 'active' : ''; ?>">التصنيفات</a>

        <a href="expenses.php" class="<?php echo $currentPage === 'expenses.php'
? 'active' : ''; ?>">المصروفات</a>

    </nav>
</div>

<div class="header-actions">
    <?php if ($loggedIn): ?>
        <span class="badge" style="background:rgba(37,99,235,.08);
color:#1d4ed8;">
            <?php echo htmlspecialchars($userName); ?>
        </span>
        <a href="profile.php" class="btn btn-secondary">الملف الشخصي</a>
        <a href="logout.php" class="btn btn-danger" data-confirm="
هل تريد تسجيل
خروج؟">خروج</a>
    <?php else: ?>
        <a href="login.php" class="btn btn-primary">تسجيل الدخول</a>
    <?php endif; ?>
</div>
</div>
</header>

<main>

```

لماذا نستخدمه؟

لأنه يجعل كل الصفحات متشابهة من حيث التصميم والتنقل، فلا يشعر المستخدم أن كل صفحة من عالم مختلف.

مثال للفائدة

بدل أن نكتب قائمة التنقل في كل صفحة، نضعها مرة واحدة داخل header.php ثم نستدعي الملف في جميع الصفحات.

ملف footer.php

هذا الملف يضم نهاية الصفحة.

يتم إنشاؤه داخل مجلد **includes** فيه نضع العناصر التي تظهر في أسفل الموقع، مثل حقوق النشر أو بعض الروابط أو السكريبتات النهائية.

الفائدة

يسهل على المبتدئ تعديل النهاية في مكان واحد بدل تكرارها في كل الصفحات.

Footer.php

```
</main>

<footer class="footer">
  <div class="container footer-top">
    <div>
      <div class="footer-brand">
        <div class="brand-mark" aria-hidden="true">
          <svg viewBox="0 0 24 24"><path d="M12 2.5 3 7v10l9 4.5 9-4.5V7l-9-4.5zm0 2.24L18.82 8 12 11.27 5.18 8 4.74zM5 9.62l6 3v7.86l-6-3V9.62zm14 0v7.86l-6 3v-7.86l6-3z"/></svg>
        </div>
        <div>
          <h4>Expense Tracker</h4>
          <p>واجهة حديثة لإدارة المصروفات والتقارير والميزانية بكل سهولة</p>
        </div>
      </div>
    </div>
  </div>
  <div>
    <h4>روابط سريعة</h4>
    <div class="footer-links">
      <a href="dashboard.php">لوحة التحكم</a>
      <a href="categories.php">التصنيفات</a>
      <a href="expenses.php">المصروفات</a>
      <a href="reports.php">التقارير</a>
    </div>
  </div>
</div>
```

```

</div>
<div>
  <h4>نبذة</h4>
  <p>
يساعدك هذا النظام على متابعة نفقاتك اليومية، وعرض ملخصات دقيقة، واتخاذ قرارات مالية
أفضل من
  خلال بيانات منظمة وتقارير واضحة.
  </p>
</div>
</div>
<div class="container footer-bottom">
  <div>© <span data-year></span> Expense Tracker. جميع الحقوق محفوظة.</div>
  <div>تصميم أنيق، متجاوب، وداعم للغة العربية</div>
</div>
</footer>
</body>
</html>

```

```

<div>
  <h4>نبذة</h4>
  <p>
يساعدك هذا النظام على متابعة نفقاتك اليومية، وعرض ملخصات دقيقة، واتخاذ قرارات مالية
أفضل من
  خلال بيانات منظمة وتقارير واضحة.
  </p>
</div>
</div>
<div class="container footer-bottom">
  <div>© <span data-year></span> Expense Tracker. جميع الحقوق محفوظة.</div>
  <div>تصميم أنيق، متجاوب، وداعم للغة العربية</div>
</div>

```

```
</footer>
</body>
</html>
```

ملف auth.php

ملف auth.php يُستخدم لحماية الصفحات التي تتطلب تسجيل دخول. يبدأ الملف بتشغيل الجلسة، ثم يفحص وجود user_id داخل \$_SESSION. إذا لم يكن المستخدم مسجل الدخول، يتم تحويله إلى صفحة تسجيل الدخول. كما يحدد اسم الصفحة الحالية لاستخدامه في تمييز الرابط النشط في الهيدر، ويجهز متغيرات مثل حالة الدخول واسم المستخدم لعرضها داخل الواجهة.

يتم انشاء ملف auth.php داخل مجلد includes

auth.php

```
<?php
session_start();

if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

$currentPage = basename($_SERVER['PHP_SELF']);
$loggedIn = isset($_SESSION['user_id']);
$userName = $_SESSION['user_name'] ?? 'المستخدم';
?>
```

ملف style.css

هذا الملف هو المسؤول عن تنسيق شكل المشروع بالكامل. بدونه ستكون الصفحات عادية جدًا وغير مرتبة.

يتم انشاء ملف style.css داخل assets/css

ماذا يمكن أن يحتوي؟

- لون الخلفية.
- تنسيق البطاقات.
- تنسيق الأزرار.

- تنسيق الجداول.
- تنسيق النصوص العربية.
- ضبط المسافات والهوامش.

لماذا هو مهم؟

لأنه يجعل المشروع جميلاً وسهل القراءة، وهذا مهم جداً في المشاريع التعليمية ومشاريع التخرج.

ملاحظة للمبتدئين

من الأفضل أن نستخدم ألواناً متناسقة وواضحة، وأن نجعل النص العربي مقروءاً بشكل جيد، خصوصاً في المشاريع التي تعتمد على اللغة العربية.

styles.php

```
:root{
  --bg:#f4f7fb;
  --surface:#ffffff;
  --surface-2:#f8fbff;
  --text:#1e293b;
  --muted:#64748b;
  --primary:#2563eb;
  --primary-dark:#1d4ed8;
  --accent:#14b8a6;
  --danger:#ef4444;
  --warning:#f59e0b;
  --success:#10b981;
  --border:rgba(148,163,184,.22);
  --shadow:0 18px 45px rgba(15,23,42,.08);
  --radius:22px;
}

*{box-sizing:border-box}

html{scroll-behavior:smooth}

body{
```

```

margin:0;
font-family:'Cairo','Tajawal','Segoe UI',Tahoma,sans-serif;
background:
  radial-gradient(circle at top left, rgba(37,99,235,.12), transparent 28%),
  radial-gradient(circle at top right, rgba(20,184,166,.10), transparent 24%),
  linear-gradient(180deg,#f8fbff 0%, #f4f7fb 100%);
color:var(--text);
direction:rtl;
text-align:right;
min-height:100vh;
}

a{
  color:inherit;
  text-decoration:none;
}

img{
  max-width:100%;
  display:block;
}

.container{
  width:min(1180px,calc(100% - 32px));
  margin-inline:auto;
}

/* عناوين عامة */
h2{
  margin:0 0 14px;
  font-size:clamp(1.5rem, 2.5vw, 3rem);
  line-height:1.5;

```

```
font-weight:800;
color:#fff;
text-align:center;
background:linear-gradient(135deg, rgba(37,99,235,.94), rgba(20,184,166,.92));
text-shadow:0 4px 14px rgba(0,0,0,.18);
}

/* الهيدر */
.site-header{
  position:sticky;
  top:0;
  z-index:1000;
  backdrop-filter:blur(18px);
  background:rgba(255,255,255,.78);
  border-bottom:1px solid rgba(148,163,184,.18);
}

.topbar{
  display:flex;
  align-items:center;
  justify-content:space-between;
  gap:18px;
  padding:14px 0;
}

.brand{
  display:flex;
  align-items:center;
  gap:14px;
  min-width:220px;
  flex-shrink:0;
}
```



```
.brand-mark{
  width:52px;
  height:52px;
  border-radius:16px;
  background:linear-gradient(135deg,var(--primary),var(--accent));
  display:grid;
  place-items:center;
  color:#fff;
  box-shadow:0 10px 28px rgba(37,99,235,.28);
  flex:0 0 auto;
}

.brand-mark svg{
  width:28px;
  height:28px;
  fill:currentColor;
}

.brand-text{
  min-width:0;
}

.brand-text h1{
  margin:0;
  font-size:1.08rem;
  line-height:1.2;
}

.brand-text p{
  margin:4px 0 0;
  color:var(--muted);
}
```

```
font-size:.92rem;
}

/* التنقل */
.nav-wrap{
  flex:1;
  display:flex;
  justify-content:center;
  align-items:center;
  min-width:0;
}

.nav-toggle{
  display:none;
  width:44px;
  height:44px;
  border:none;
  border-radius:14px;
  background:var(--surface);
  box-shadow:var(--shadow);
  cursor:pointer;
}

.nav-toggle span,
.nav-toggle span::before,
.nav-toggle span::after{
  display:block;
  width:20px;
  height:2px;
  background:var(--text);
  border-radius:999px;
  position:relative;
```

```
margin-inline:auto;
}

.nav-toggle span::before,
.nav-toggle span::after{
  content:"";
  position:absolute;
  inset-inline-start:0;
}

.nav-toggle span::before{top:-6px}
.nav-toggle span::after{top:6px}

.main-nav{
  display:flex;
  align-items:center;
  justify-content:center;
  gap:2px;
  flex-wrap:nowrap;
  white-space:nowrap;
}

.main-nav a{
  padding:11px 12px;
  border-radius:999px;
  color:var(--muted);
  font-weight:800;
  font-size:.98rem;
  line-height:1;
  transition:.25s ease;
  background:transparent;
  border:1px solid transparent;
```

```

}

.main-nav a:hover{
  background:rgba(37,99,235,.08);
  color:var(--primary-dark);
  border-color:rgba(37,99,235,.10);
}

.main-nav a.active{
  background:linear-gradient(135deg, rgba(37,99,235,.12), rgba(20,184,166,.10));
  color:var(--primary-dark);
  border-color:rgba(37,99,235,.14);
  box-shadow:0 8px 18px rgba(37,99,235,.08);
}

/* إجراءات الهيدر */
.header-actions{
  display:flex;
  align-items:center;
  gap:10px;
  flex-shrink:0;
}

.badge{
  display:inline-flex;
  align-items:center;
  gap:8px;
  padding:9px 14px;
  border-radius:999px;
  background:rgba(255,255,255,.14);
  color:#fff;
  font-weight:700;
}

```

```

font-size:.92rem;
}

.header-actions .badge,
.header-badge{
background:linear-gradient(135deg, rgba(37,99,235,.08), rgba(20,184,166,.08));
color:#1d4ed8;
padding:10px 14px;
border-radius:999px;
font-weight:800;
font-size:.95rem;
border:1px solid rgba(37,99,235,.10);
white-space:nowrap;
}

/* الأزرار */
.btn,
button.btn,
a.btn,
input[type='submit'],
input[type='button']{
display:inline-flex;
align-items:center;
justify-content:center;
gap:8px;
border:none;
border-radius:16px;
padding:12px 18px;
font-weight:800;
cursor:pointer;
transition:all .25s ease;
box-shadow:none;

```

```
}

.btn-primary{
  background:linear-gradient(135deg,var(--primary),#3b82f6);
  color:#fff;
}

.btn-primary:hover{
  transform:translateY(-1px);
  filter:saturate(1.05);
}

.btn-secondary{
  background:#e2e8f0;
  color:#0f172a;
}

.btn-secondary:hover{
  background:#cbd5e1;
}

.btn-success{
  background:linear-gradient(135deg,var(--success),#22c55e);
  color:#fff;
}

.btn-danger{
  background:linear-gradient(135deg,var(--danger),#f97316);
  color:#fff;
}

.btn-warning{
  background:linear-gradient(135deg,var(--warning),#fbbf24);
```

```
color:#111827;
}

.header-actions .btn{
padding:11px 16px;
border-radius:999px;
font-size:.95rem;
font-weight:800;
min-height:44px;
}

.header-actions .btn-secondary{
background:#eef2f7;
color:#0f172a;
border:1px solid rgba(148,163,184,.18);
}

.header-actions .btn-secondary:hover{
background:#e2e8f0;
}

.header-actions .btn-danger{
box-shadow:0 10px 24px rgba(239,68,68,.18);
}

/*الهيرو*/
.hero{
position:relative;
overflow:hidden;
margin:28px 0 26px;
padding:42px 0;
}
```

```
.hero-card{
  position:relative;
  background:linear-gradient(135deg, rgba(37,99,235,.94), rgba(20,184,166,.92));
  color:#fff;
  border-radius:28px;
  padding:40px;
  box-shadow:0 22px 60px rgba(37,99,235,.20);
  overflow:hidden;
}

.hero-card::before,
.hero-card::after{
  content:"";
  position:absolute;
  border-radius:999px;
  background:rgba(255,255,255,.11);
}

.hero-card::before{
  width:220px;
  height:220px;
  top:-90px;
  left:-70px;
}

.hero-card::after{
  width:180px;
  height:180px;
  bottom:-70px;
  right:-50px;
}
```



```
.hero-grid{
  position:relative;
  display:grid;
  grid-template-columns:1.3fr .9fr;
  gap:24px;
  align-items:center;
}

.hero h2{
  margin:0 0 14px;
  font-size:clamp(2rem,4vw,3.4rem);
  line-height:1.15;
  background:none;
  text-align:right;
}

.hero p{
  margin:0 0 22px;
  max-width:62ch;
  font-size:1.05rem;
  line-height:1.9;
  color:rgba(255,255,255,.92);
}

.hero-badges{
  display:flex;
  flex-wrap:wrap;
  gap:10px;
}

.hero-panel{
```

```

background:rgba(255,255,255,.12);
border:1px solid rgba(255,255,255,.18);
border-radius:24px;
padding:22px;
backdrop-filter:blur(10px);
}

.hero-panel .mini-card{
  display:flex;
  align-items:center;
  justify-content:space-between;
  gap:16px;
  padding:14px 0;
  border-bottom:1px solid rgba(255,255,255,.15);
}

.hero-panel .mini-card:last-child{
  border-bottom:none;
}

.hero-panel .mini-card strong{
  font-size:1.05rem;
}

.hero-panel .mini-card span{
  color:rgba(255,255,255,.84);
}

/* الأقسام */
.section{
  margin:28px 0 34px;
}

```

```
.section-head{  
  display:flex;  
  align-items:end;  
  justify-content:space-between;  
  gap:18px;  
  margin-bottom:18px;  
}
```

```
.section-head h3{  
  margin:0;  
  font-size:1.45rem;  
}
```

```
.section-head p{  
  margin:0;  
  color:var(--muted);  
}
```

```
/* الكروت */  
.card,  
.panel,  
.content-box,  
.table-wrap{  
  background:var(--surface);  
  border:1px solid var(--border);  
  border-radius:var(--radius);  
  box-shadow:var(--shadow);  
}
```

```
.card{  
  padding:22px;
```

```
}

.grid-3{
  display:grid;
  grid-template-columns:repeat(3,1fr);
  gap:18px;
}

.grid-4{
  display:grid;
  grid-template-columns:repeat(4,1fr);
  gap:18px;
}

.stat{
  position:relative;
  overflow:hidden;
  padding:22px;
  border-radius:24px;
  background:linear-gradient(180deg,#fff, #f8fbff);
  border:1px solid rgba(148,163,184,.16);
  box-shadow:var(--shadow);
}

.stat::before{
  content:"";
  position:absolute;
  inset-inline-end:-30px;
  top:-20px;
  width:110px;
  height:110px;
  border-radius:50%;
}
```

```
background:radial-gradient(circle, rgba(37,99,235,.13), transparent 70%);
}

.stat .label{
  color:var(--muted);
  font-size:.94rem;
  font-weight:700;
}

.stat .value{
  margin-top:10px;
  font-size:1.8rem;
  font-weight:900;
  letter-spacing:.2px;
}

.stat .hint{
  margin-top:8px;
  color:var(--muted);
  font-size:.92rem;
}

.form-card{
  padding:24px;
}

/* النماذج */
label{
  display:block;
  margin-bottom:8px;
  font-weight:800;
  color:#0f172a;
```

```
}

input[type='text'],
input[type='email'],
input[type='password'],
input[type='date'],
input[type='number'],
select,
textarea{
  width:100%;
  padding:13px 16px;
  border-radius:16px;
  border:1px solid #dbe3ee;
  background:#fff;
  color:var(--text);
  font-family:inherit;
  font-size:1rem;
  outline:none;
  transition:.2s ease;
}

input:focus,
select:focus,
textarea:focus{
  border-color:rgba(37,99,235,.55);
  box-shadow:0 0 0 4px rgba(37,99,235,.12);
}

textarea{
  min-height:120px;
  resize:vertical;
}
```

```
/* الجداول */
.table-wrap{
  overflow:hidden;
}

table{
  width:100%;
  border-collapse:collapse;
}

th,
td{
  padding:14px 16px;
  border-bottom:1px solid #e8eef5;
  text-align:right;
  vertical-align:top;
}

th{
  background:#f8fbff;
  color:#0f172a;
  font-size:.92rem;
  letter-spacing:.2px;
}

tbody tr:hover{
  background:#fbfdff;
}

/* التنبيهات */
.alert{
```

```
padding:14px 16px;
border-radius:16px;
font-weight:700;
}

.alert-success{
  background:#ecfdf5;
  color:#065f46;
  border:1px solid #a7f3d0;
}

.alert-danger{
  background:#fef2f2;
  color:#991b1b;
  border:1px solid #fecaca;
}

.alert-warning{
  background:#fffbeb;
  color:#92400e;
  border:1px solid #fde68a;
}

.alert-info{
  background:#eff6ff;
  color:#1e40af;
  border:1px solid #bfdbfe;
}

/* الفوتر */
.footer{
  margin-top:40px;
```



```
background:linear-gradient(180deg,#0f172a,#111827);
color:#cbd5e1;
}

.footer-top{
display:grid;
grid-template-columns:1.2fr .9fr .9fr;
gap:24px;
padding:42px 0 24px;
}

.footer h4{
margin:0 0 14px;
color:#fff;
font-size:1.05rem;
}

.footer p{
margin:0;
color:#cbd5e1;
line-height:1.9;
}

.footer-links{
display:grid;
gap:10px;
}

.footer-links a{
color:#cbd5e1;
transition:.2s ease;
}
```

```
.footer-links a:hover{
  color:#fff;
}

.footer-bottom{
  border-top:1px solid rgba(148,163,184,.16);
  padding:16px 0 22px;
  display:flex;
  align-items:center;
  justify-content:space-between;
  gap:12px;
  font-size:.94rem;
}

.footer-brand{
  display:flex;
  align-items:center;
  gap:12px;
}

.footer-brand .brand-mark{
  width:44px;
  height:44px;
  border-radius:14px;
}

/* KPI */
.kpi-list{
  display:grid;
  grid-template-columns:repeat(2,1fr);
  gap:14px;
```

```
}

.kpi{
  background:#f8fbff;
  border:1px solid rgba(148,163,184,.18);
  border-radius:18px;
  padding:16px;
}

.kpi strong{
  display:block;
  font-size:1.2rem;
}

.kpi span{
  color:var(--muted);
  font-size:.92rem;
}

/* عناصر زخرفية */
.hero-illustration{
  display:grid;
  place-items:center;
  min-height:280px;
}

.hero-illustration .bubble{
  width:min(320px,100%);
  aspect-ratio:1;
  border-radius:32px;
  background:rgba(255,255,255,.12);
  border:1px solid rgba(255,255,255,.16);
}
```

```
position:relative;
overflow:hidden;
}

.hero-illustration .bubble::before,
.hero-illustration .bubble::after{
  content:"";
  position:absolute;
  border-radius:50%;
  background:rgba(255,255,255,.12);
}

.hero-illustration .bubble::before{
  width:170px;
  height:170px;
  top:22px;
  right:22px;
}

.hero-illustration .bubble::after{
  width:130px;
  height:130px;
  bottom:18px;
  left:18px;
}

.hero-icon{
  position:absolute;
  inset:0;
  display:grid;
  place-items:center;
  color:#fff;
```

```
}

.hero-icon svg{
  width:120px;
  height:120px;
  fill:currentColor;
  opacity:.96;
}

.logo-inline{
  display:inline-flex;
  align-items:center;
  gap:10px;
  font-weight:900;
}

.logo-inline .brand-mark{
  width:38px;
  height:38px;
  border-radius:12px;
}

.hide-mobile{
  display:block;
}

/*الكروت الملونة*/
.card.card-colored{
  color:#fff;
  border:none;
  box-shadow:0 18px 35px rgba(15, 23, 42, 0.12);
}
```

```
.card.card-colored h2,  
.card.card-colored h3,  
.card.card-colored h4,  
.card.card-colored p,  
.card.card-colored span,  
.card.card-colored strong,  
.card.card-colored div,  
.card.card-colored li,  
.card.card-colored label{  
  color:inherit;  
}  
  
.card.card-blue{  
  background:linear-gradient(135deg, #2563eb, #3b82f6);  
}  
  
.card.card-green{  
  background:linear-gradient(135deg, #10b981, #34d399);  
}  
  
.card.card-orange{  
  background:linear-gradient(135deg, #f59e0b, #fbbf24);  
  color:#111827;  
}  
  
.card.card-red{  
  background:linear-gradient(135deg, #ef4444, #f87171);  
}  
  
.card.card-purple{  
  background:linear-gradient(135deg, #7c3aed, #a78bfa);  
}
```

```
.card.card-teal{
  background:linear-gradient(135deg, #14b8a6, #2dd4bf);
}

.card.card-sky{
  background:linear-gradient(135deg, #0ea5e9, #38bdf8);
}

.card.card-pink{
  background:linear-gradient(135deg, #ec4899, #f472b6);
}

.card.card-gray{
  background:linear-gradient(135deg, #6b7280, #9ca3af);
  color:#f9fafb;
}

/* الشاشات المتوسطة */
@media (max-width: 1100px){
  .topbar{
    gap:12px;
  }

  .brand{
    min-width:auto;
  }

  .brand-text h1{
    font-size:1rem;
  }
}
```

```
.brand-text p{
  font-size:.85rem;
}

.main-nav a{
  padding:10px 12px;
  font-size:.92rem;
}

.header-actions .badge{
  display:none;
}

.header-actions .btn{
  padding:10px 14px;
  font-size:.9rem;
}
}

/* الشاشات أقل من 992 */
@media (max-width: 992px){
  .hero-grid,
  .footer-top,
  .grid-3,
  .grid-4{
    grid-template-columns:1fr 1fr;
  }

  .section-head{
    flex-direction:column;
    align-items:flex-start;
  }
}
```



```
}

/* الجوال */
@media (max-width: 768px){
  .topbar{
    padding:12px 0;
  }

  .nav-toggle{
    display:inline-grid;
    place-items:center;
  }

  .nav-wrap{
    position:relative;
    flex:0 0 auto;
  }

  .main-nav{
    position:absolute;
    top:calc(100% + 12px);
    inset-inline-end:0;
    min-width:min(320px,calc(100vw - 24px));
    padding:12px;
    background:rgba(255,255,255,.97);
    border:1px solid rgba(148,163,184,.18);
    border-radius:20px;
    box-shadow:var(--shadow);
    display:none;
    flex-direction:column;
    align-items:stretch;
    gap:8px;
  }
}
```

```
white-space:normal;
}

.main-nav.is-open{
  display:flex;
}

.main-nav a{
  width:100%;
  border-radius:14px;
  padding:13px 14px;
  text-align:right;
}

.header-actions{
  gap:8px;
}

.header-actions .btn{
  padding:10px 14px;
}

.hero{
  padding:18px 0 28px;
}

.hero-card{
  padding:26px;
}

.hero-grid,
.footer-top,
```

```
.grid-3,  
.grid-4{  
  grid-template-columns:1fr;  
}  
  
.hero h2{  
  font-size:2rem;  
}  
  
.footer-bottom{  
  flex-direction:column;  
  align-items:flex-start;  
}  
  
.kpi-list{  
  grid-template-columns:1fr;  
}  
  
.hide-mobile{  
  display:none;  
}  
}  
  
/* الشاشات الصغيرة جدًا */  
@media (max-width: 560px){  
  .container{  
    width:min(100% - 20px,1180px);  
  }  
  
  .card,  
  .form-card,  
  .stat,
```

```
.hero-card{
  border-radius:20px;
}

th,
td{
  padding:12px 10px;
  font-size:.95rem;
}

.header-actions .btn{
  min-height:40px;
  padding:10px 12px;
  font-size:.88rem;
}

.brand-mark{
  width:46px;
  height:46px;
  border-radius:14px;
}

.brand-mark svg{
  width:24px;
  height:24px;
}
}
```

ملف script.js

هذا الملف مخصص للجافاسكربت، ويمكن استخدامه لتحسين التفاعل داخل الموقع.

يتم إنشاء هذا الملف في **assets/script**
أمثلة على استخدامه

- إظهار رسالة تأكيد قبل الحذف.
- إظهار تنبيه للمستخدم.
- التحكم في بعض العناصر التفاعلية.

هل هو ضروري من البداية؟

ليس ضروريًا جدًا في البداية، لكن من الأفضل إنشاءه من البداية حتى يكون المشروع جاهزًا لإضافة التفاعلات لاحقًا.

Script.js

```
document.addEventListener('DOMContentLoaded', () => {
  const navToggle = document.querySelector('[data-nav-toggle]');
  const navMenu = document.querySelector('[data-nav-menu]');
  const yearNodes = document.querySelectorAll('[data-year]');

  if (navToggle && navMenu) {
    navToggle.addEventListener('click', () => {
      navMenu.classList.toggle('is-open');
      navToggle.setAttribute('aria-expanded', navMenu.classList.contains('is-open') ? 'true' : 'false');
    });

    document.addEventListener('click', (e) => {
      if (!navMenu.contains(e.target) && !navToggle.contains(e.target)) {
        navMenu.classList.remove('is-open');
        navToggle.setAttribute('aria-expanded', 'false');
      }
    });
  }

  yearNodes.forEach(node => node.textContent = new Date().getFullYear());

  document.querySelectorAll('[data-confirm]').forEach(link => {
```

```

link.addEventListener('click', (e) => {
  const message = link.getAttribute('data-confirm') || 'هل أنت متأكد؟';
  if (!confirm(message)) e.preventDefault();
});

document.querySelectorAll('[data-smooth]').forEach(anchor => {
  anchor.addEventListener('click', function (e) {
    const target = document.querySelector(this.getAttribute('href'));
    if (target) {
      e.preventDefault();
      target.scrollIntoView({ behavior: 'smooth', block: 'start' });
    }
  });
});
});

```

كيف نستدعي الملفات العامة؟

في الصفحات التي نريد أن نظهر فيها الهيدر، نكتب مثلاً:

php

```
<?php include 'header.php'; ?>
```

وفي نهاية الصفحة نضيف:

php

```
<?php include 'footer.php'; ?>
```

بهذا الشكل نستخدم نفس التصميم في كل الصفحات بسهولة.

شكل تنظيم المجلدات

يمكن أن يكون تنظيم المشروع كالتالي:

- includes/
- config.php
- header.php
- footer.php
- assets/
- css/style.css
- js/script.js
- categories.php
- expenses.php
- reports.php
- budget.php
- login.php
- register.php
- -----

هذا التنظيم يجعل المشروع مرتبًا ويسهل الرجوع إلى أي ملف بسرعة.

لماذا هذا التنظيم مهم جدًا؟

لأنه يساعد على:

- تقليل تكرار الكود.
- سهولة الصيانة.
- سهولة التعديل.
- سهولة التوسعة مستقبلاً.
- جعل المشروع أكثر احترافية.

مثال بسيط

إذا أردنا تغيير اسم الموقع أو تعديل القائمة، فسنعدل header.php فقط، وستظهر التغييرات في كل الصفحات تلقائيًا. وهذا أفضل بكثير من تعديل كل صفحة يدويًا.

ملاحظات مهمة للمبتدئين

- لا تضعي كل شيء في ملف واحد.
- اجعلي كل ملف له وظيفة محددة.
- احرصي على تسمية الملفات بشكل واضح.
- لا تكرري نفس الأكواد في كل الصفحات.
- جربي فتح كل ملف بعد إضافته للتأكد من أنه يعمل.

خلاصة الدرس

في هذا الدرس تعلمنا كيف ننظم هيكل المشروع من الداخل. عرفنا وظيفة الملفات العامة مثل `header.php` و `footer.php` و `style.css` و `script.js`، وتعلمنا لماذا نستخدمها في جميع الصفحات. كما عرفنا أن التنظيم الجيد من البداية يجعل المشروع أسهل في التطوير والتعديل، ويمنحه شكلاً احترافياً ومريحاً للمستخدم.

الدرس الخامس : ملف الاتصال بقاعدة البيانات

مقدمة الدرس

بعد أن أنشأنا قاعدة البيانات والجداول، والهيكل العام للمشروع، نأتي الآن إلى خطوة مهمة جدًا وهي **الاتصال بقاعدة البيانات داخل PHP**. هذه الخطوة هي الجسر الذي يربط صفحات المشروع بقاعدة البيانات، وبدونها لن نستطيع قراءة البيانات أو حفظها أو تعديلها أو حذفها. في هذا الدرس سننشئ ملفًا خاصًا بالاتصال، ثم نشرحه بطريقة بسيطة حتى تفهمه المبتدئات بسهولة.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- ما معنى ملف الاتصال بقاعدة البيانات.
- لماذا نضعه في ملف مستقل.
- كيف نكتب الكود الصحيح للاتصال.
- كيف نختبر أن الاتصال يعمل بشكل سليم.
- كيف نستخدمه داخل بقية الصفحات.

ما هو ملف الاتصال؟

ملف الاتصال هو ملف PHP يحتوي على بيانات الدخول إلى قاعدة البيانات، مثل:

- اسم السيرفر.
- اسم المستخدم.
- كلمة المرور.
- اسم قاعدة البيانات.

عادة نسمي هذا الملف:

- config.php
- هذا الملف مهم جدًا لأن جميع الصفحات التي تتعامل مع البيانات ستحتاج إليه.

يتم إنشاء ملف config.php داخل هذا مجلد includes

لماذا نستخدم ملفًا مستقلًا؟

من الأفضل ألا نكتب كود الاتصال داخل كل صفحة، لأن ذلك سيجعل المشروع غير مرتب. عندما نضع كود الاتصال في ملف واحد فقط، يصبح من السهل:

- تعديل البيانات لاحقًا.
- إعادة استخدام نفس الاتصال في كل الصفحات.
- تقليل الأخطاء والتكرار.

فكرة عمله

عند فتح أي صفحة تحتاج إلى قاعدة البيانات، نقوم أولاً باستدعاء ملف الاتصال. وبعد ذلك تصبح الصفحة قادرة على تنفيذ استعلامات مثل:

- إضافة سجل جديد.
- عرض بيانات.
- تعديل بيانات.
- حذف بيانات.

كود ملف الاتصال

فيما يلي كود ملف الاتصال الكامل باستخدام MySQLi:

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "expense_tracker_db";

$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
$conn->set_charset("utf8mb4");
?>
```

شرح الكود سطرًا سطرًا

(1) تعريف بيانات الاتصال

```
$servername = "localhost";  
$username = "root";  
$password = "";  
$dbname = "expense_tracker_db";
```

هنا نحدد معلومات قاعدة البيانات:

- localhost يعني أن قاعدة البيانات تعمل على نفس الجهاز.
- root هو اسم المستخدم الافتراضي غالبًا في XAMPP.
- كلمة المرور غالبًا تكون فارغة في البداية.
- expense_tracker_db هو اسم قاعدة البيانات التي أنشأناها.

(2) إنشاء الاتصال

```
$conn = new mysqli($servername, $username, $password, $dbname);
```

هذا السطر ينشئ اتصالًا جديدًا بقاعدة البيانات باستخدام MySQLi.

(3) التحقق من نجاح الاتصال

```
if ($conn->connect_error) {  
    die("Connection failed: " . $conn->connect_error);  
}
```

إذا حدث خطأ في الاتصال، يتوقف البرنامج ويظهر السبب. هذا مهم جدًا حتى نعرف أين المشكلة بدلًا من أن تظهر الصفحة فارغة.

(4) دعم اللغة العربية

```
$conn->set_charset("utf8mb4");
```

هذا السطر مهم جدًا لأنه يجعل قاعدة البيانات تدعم اللغة العربية بشكل صحيح، ويمنع ظهور الحروف بشكل غير مفهوم.

لماذا اخترنا utf8mb4 ؟

لأن هذا الترميز هو الأفضل لحفظ النصوص العربية والرموز الخاصة. وبما أن مشروعنا سيستخدم العربية في كثير من الأسماء والوصف، فمن الضروري جدًا ضبط الترميز من البداية.

اختبار ملف الاتصال

بعد إنشاء الملف، من الأفضل أن نختبره للتأكد من أنه يعمل بشكل صحيح. يمكن إنشاء ملف جديد مثل test.php ونكتب فيه:

```
<?php
include 'config.php';
echo "Connection successful";
?>
```

إذا ظهرت العبارة Connection successful فهذا يعني أن الاتصال يعمل بشكل صحيح.

كيف نستخدم ملف الاتصال داخل الصفحات؟

في أي صفحة تحتاج إلى قاعدة البيانات، نضيف في البداية:

```
<?php
include 'config.php';
?>
```

وبعدها نستطيع تنفيذ أوامر SQL داخل الصفحة.

خلاصة الدرس

في هذا الدرس أنشأنا ملف الاتصال بقاعدة البيانات وتعلمنا كيف نربط PHP مع MySQL بطريقة صحيحة. كما تعرفنا على أهمية استخدام utf8mb4 لدعم اللغة العربية، وعلى كيفية اختبار الاتصال واستخدامه داخل باقي صفحات المشروع. هذا الملف هو الأساس الذي سنبنى عليه جميع الدروس القادمة.

الدرس السادس: التسجيل وإنشاء حساب جديد

مقدمة الدرس

هذه الصفحة تسمح للمستخدم بإنشاء حسابه الخاص داخل النظام، حتى يتمكن لاحقاً من تسجيل الدخول وإدارة مصروفاته بشكل مستقل وآمن.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيف ننشئ صفحة تسجيل جديدة.
- كيف نجمع بيانات المستخدم داخل نموذج.
- كيف نخزن البيانات في جدول users.
- كيف نشفر كلمة المرور بشكل صحيح.
- كيف نوجه المستخدم إلى صفحة الدخول بعد التسجيل.

لماذا نحتاج صفحة التسجيل؟

في أنظمة إدارة المصروفات، من المهم أن يكون لكل مستخدم حسابه الخاص. صفحة التسجيل تساعد على:

- إنشاء حساب جديد بسهولة.
- حفظ بيانات المستخدم بشكل منفصل.
- ربط المصروفات والميزانية بالمستخدم نفسه.
- حماية البيانات الشخصية.

فكرة الصفحة

عندما يفتح المستخدم صفحة التسجيل، يملأ البيانات التالية:

- الاسم الكامل.
- البريد الإلكتروني.
- كلمة المرور.
- تأكيد كلمة المرور.

ثم يضغط على زر التسجيل، فيتحقق النظام من صحة البيانات، وإذا كانت صحيحة يتم حفظها في قاعدة البيانات، وبعدها ينتقل المستخدم إلى صفحة تسجيل الدخول.

الخطوات العملية للدرس

الخطوة الأولى: إنشاء صفحة التسجيل

سننشئ صفحة اسمها:

register.php

الخطوة الثالثة: إنشاء النموذج

النموذج سيحتوي على:

- الاسم الكامل.
- البريد الإلكتروني.
- كلمة المرور.
- تأكيد كلمة المرور.

الخطوة الرابعة: التحقق من صحة البيانات

سنفحص:

- هل الحقول فارغة؟
- هل كلمة المرور وتأكيدها متطابقتان؟
- هل البريد الإلكتروني موجود مسبقاً؟

الخطوة الخامسة: حفظ البيانات

إذا كانت البيانات صحيحة، نحفظها في الجدول بعد تشفير كلمة المرور.

الخطوة السادسة: الانتقال إلى صفحة الدخول

بعد نجاح التسجيل، نوجه المستخدم إلى صفحة login.php.

كود صفحة التسجيل

```
<?php
include 'includes/config.php';
include 'includes/header.php';
?>

<div class="container" style="margin-top:30px;">
```

```

<div class="card">

    <h2>إدارة التصنيفات</h2>

    <p>من هنا يمكنك عرض جميع التصنيفات وإدارتها بسهولة</p>

    <a href="add_category.php" class="btn btn-primary">إضافة تصنيف جديد</a>

    <table>

        <thead>

            <tr>

                <th>م</th>

                <th>اسم التصنيف</th>

                <th>الإجراءات</th>

            </tr>

        </thead>

        <tbody>

            <?php

                $sql = "SELECT * FROM categories ORDER BY id DESC";

                $result = $conn->query($sql);

                if ($result && $result->num_rows > 0) {

                    $i = 1;

                    while ($row = $result->fetch_assoc()) {

                        echo "<tr>";

                        echo "<td>" . $i . "</td>";

                        echo "<td>" . htmlspecialchars($row['name']) . "</td>";

                        echo "<td>

                            <a href='edit_category.php?id=" . $row['id'] . "'" class='btn btn-
warning'>تعديل</a>

                            <a href='delete_category.php?id=" . $row['id'] . "'" class='btn btn-
danger' onclick=\"return confirm('هل أنت متأكد من الحذف؟')\">حذف</a>


```

```

        </td>";
        echo "</tr>";
        $i++;
    }
} else {
    echo "<tr><td colspan='3'>لا توجد تصنيفات حتى الآن</td></tr>";
}
?>
</tbody>
</table>
</div>
</div>

<?php include 'includes/footer.php'; ?>

```

شرح الكود

1) تضمين ملف الاتصال

```
include 'includes/config.php';
```

نحتاج إلى قاعدة البيانات حتى نحفظ بيانات الحساب الجديد.

2) التحقق من الضغط على زر التسجيل

```
if (isset($_POST['register'])) {
```

هذا يعني أن المستخدم أرسل النموذج ام لا

3) قراءة البيانات

```
$full_name = trim($_POST['full_name']);
```

```
$email = trim($_POST['email']);
```

```
$password = trim($_POST['password']);
```

```
$confirm_password = trim($_POST['confirm_password']);
```

نأخذ القيم من النموذج وننظفها من الفراغات الزائدة.

4) التحقق من أن الحقول ليست فارغة

لا يمكن إنشاء حساب بدون بيانات أساسية.

5) التحقق من تطابق كلمة المرور

```
if ($password === $confirm_password)
```

هذا مهم حتى لا يسجل المستخدم كلمة مرور مختلفة عن التي يريد.

6) التأكد من أن البريد غير مكرر

```
SELECT id FROM users WHERE email = ?
```

هذا يمنع إنشاء أكثر من حساب بنفس البريد الإلكتروني.

7) تشفير كلمة المرور

```
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);
```

هذا السطر مهم جدًا لأنه يحفظ كلمة المرور بشكل مشفر وآمن.

8) حفظ الحساب في قاعدة البيانات

```
INSERT INTO users (full_name, email, password) VALUES (?, ?, ?)
```

بهذا يتم إنشاء الحساب الجديد.

9) تحويل المستخدم إلى صفحة الدخول

```
header("Location: login.php");
```

بعد نجاح التسجيل، ينتقل المستخدم إلى صفحة الدخول ليستعمل حسابه الجديد.

لماذا التشفير مهم؟

لا يجب أبدًا حفظ كلمة المرور كنص واضح داخل قاعدة البيانات. التشفير يحمي الحسابات من أي وصول غير مرغوب فيه، ويجعل النظام أكثر أمانًا واحترافية.

ماذا يحدث بعد التسجيل؟

بعد أن ينشئ المستخدم حسابه:

١. يتم حفظ بياناته في الجدول.

٢. ينتقل إلى صفحة تسجيل الدخول.

٣. يدخل بريده وكلمة مروره.

٤. يبدأ باستخدام لوحة التحكم والمصروفات والتقارير.

مثال عملي

إذا سجل المستخدم:

- الاسم: سارة أحمد
- البريد الإلكتروني: sarah@example.com
- كلمة المرور 123456 :

ثم أكد نفس كلمة المرور، فسيتم إنشاء الحساب بنجاح، وبعدها ينتقل إلى صفحة الدخول ليبدأ استخدام النظام.

خلاصة الدرس

في هذا الدرس تعلمنا كيف ننشئ صفحة التسجيل وإنشاء حساب جديد. تعرفنا على كيفية جمع بيانات المستخدم، والتحقق من صحتها، ومنع تكرار البريد الإلكتروني، وتشفير كلمة المرور، ثم حفظ البيانات في قاعدة البيانات. وبهذا أصبح النظام يحتوي على ميزة أساسية ومهمة جدًا تسبق تسجيل الدخول وتسمح بإنشاء حسابات جديدة بسهولة.

الدرس السابع : تسجيل الدخول والخروج

مقدمة الدرس

بعد أن أنشأنا صفحة انشاء حساب جديد، نصل الآن إلى خطوة مهمة جدًا وهي تسجيل الدخول والخروج.

هذه الخطوة ضرورية لأن النظام يجب أن يعرف من هو المستخدم الذي يعمل عليه، حتى يتم حفظ بياناته بشكل مستقل وآمن. كما أن تسجيل الدخول يسمح بحماية البيانات ومنع الوصول غير المصرح به.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيف ننشئ صفحة تسجيل الدخول.
- كيف نتحقق من اسم المستخدم وكلمة المرور.
- كيف نخزن بيانات الجلسة بعد الدخول.
- كيف ننشئ صفحة الخروج.
- كيف نحمي الصفحات بحيث لا يفتحها إلا المستخدم المسجل دخوله.

لماذا نحتاج تسجيل الدخول؟

تسجيل الدخول مهم جدًا في أي نظام مالي لأنه:

- يحافظ على الخصوصية.
 - يربط كل البيانات بمستخدم معين.
 - يمنع أي شخص آخر من مشاهدة المصروفات.
 - يسمح بعرض لوحة تحكم شخصية لكل مستخدم.
- وقد أظهرت أنظمة مشابهة لإدارة المصروفات وجود ميزة تسجيل الدخول الآمن، مع إمكانية التسجيل أو استرجاع كلمة المرور في بعض النسخ.

فكرة العمل

عندما يدخل المستخدم اسمه وكلمة المرور الصحيحة:

١. يتحقق النظام من البيانات.
٢. إذا كانت صحيحة، يتم تخزين بيانات المستخدم في الجلسة.
٣. ينتقل المستخدم إلى لوحة التحكم.
٤. إذا كانت البيانات خاطئة، تظهر رسالة خطأ.

٥. عند تسجيل الخروج يتم حذف الجلسة والعودة إلى صفحة الدخول.

الخطوات العملية للدرس

الخطوة الأولى: إنشاء صفحة تسجيل الدخول

سننشئ صفحة اسمها:

login.php

الخطوة الثالثة: التحقق من البيانات

سنقارن البريد الإلكتروني أو اسم المستخدم وكلمة المرور مع البيانات الموجودة في قاعدة البيانات.

الخطوة الرابعة: حفظ الجلسة

بعد نجاح الدخول، نخزن بيانات المستخدم داخل \$_SESSION.

الخطوة الخامسة: إنشاء صفحة الخروج

سننشئ صفحة اسمها:

logout.php

كود صفحة تسجيل الدخول login.php

```
<?php
session_start();
include 'includes/config.php';

$message = "";

if (isset($_POST['login'])) {
    $email = trim($_POST['email']);
    $password = trim($_POST['password']);

    if (!empty($email) && !empty($password)) {
```

```
$stmt = $conn->prepare("SELECT id, full_name, email, password  
FROM users WHERE email = ?");
```

```
$stmt->bind_param("s", $email);
```

```
$stmt->execute();
```

```
$result = $stmt->get_result();
```

```
$user = $result->fetch_assoc();
```

```
if ($user && password_verify($password, $user['password'])) {
```

```
    $_SESSION['user_id'] = $user['id'];
```

```
    $_SESSION['user_name'] = $user['full_name'];
```

```
    $_SESSION['user_email'] = $user['email'];
```

```
    header("Location: index.php");
```

```
    exit;
```

```
} else {
```

```
    $message = "البريد الإلكتروني أو كلمة المرور غير صحيحة";
```

```
}
```

```
$stmt->close();
```

```
} else {
```

```
    $message = "يرجى تعبئة جميع الحقول";
```

```
}
```

```
}
```

```
?>
```

```
<!DOCTYPE html>
```

```
<html lang="ar">
```

```

<head>

  <meta charset="UTF-8">

  <title>تسجيل الدخول</title>

  <link rel="stylesheet" href="assets/css/style.css">

</head>

<body>

<header class="site-header">

  <div class="container topbar">

    <div class="brand">

      <a href="dashboard.php" class="brand-mark" aria-label="العودة
للرئيسية">

        <svg viewBox="0 0 24 24" aria-hidden="true"><path d="M12
2.5 3 7v10l9 4.5 9-4.5V7l-9-4.5zm0 2.24L18.82 8 12 11.27 5.18 8 12
4.74zM5 9.62l6 3v7.86l-6-3V9.62zm14 0v7.86l-6 3v-7.86l6-3z"/></svg>

      </a>

      <div class="brand-text">

        <h1>Expense Tracker</h1>

        <p>إدارة المصروفات</p>

      </div>

    </div>

  </div>

</div>

</header>

<div class="container" style="margin-top:50px;">

  <div class="card" style="max-width:400px; margin:auto;">

    <h2>تسجيل الدخول</h2>

    <?php if (!empty($message)) : ?>

```

```

    <p style="color:red;"><?php echo $message; ?></p>
    <?php endif; ?>

    <form method="post">
        <label>البريد الإلكتروني</label>
        <input type="email" name="email" required>

        <br><br>

        <label>كلمة المرور</label>
        <input type="password" name="password" required>

        <br><br>

        <button type="submit" name="login" class="btn btn-
primary">دخول</button>
    </form>

    <!-- مباشرة style الرابط بلون مختلف - تم إضافة --!>
    <p style="text-align:center; margin-top:20px;">
        <a href="register.php" style="color:#28a745; text-
decoration:underline;">سجل الآن</a> ليس لديك حساب؟
    </p>
</div>
</div>
</body>
</html>

```

شرح كود تسجيل الدخول

(1) بدء الجلسة

```
session_start();
```

نحتاج الجلسة حتى نخزن بيانات المستخدم بعد نجاح الدخول.

(2) تضمين ملف الاتصال

```
include 'includes/config.php';
```

نحتاج قاعدة البيانات حتى نتحقق من بيانات المستخدم.

(3) قراءة البيانات

```
$email = trim($_POST['email']);
```

```
$password = trim($_POST['password']);
```

نأخذ البريد وكلمة المرور من النموذج.

(4) جلب المستخدم من قاعدة البيانات

```
SELECT id, full_name, email, password FROM users WHERE email = ?
```

نبحث عن المستخدم باستخدام البريد الإلكتروني.

(5) التحقق من كلمة المرور

```
password_verify($password, $user['password'])
```

هذا السطر مهم جدًا لأنه يقارن كلمة المرور المدخلة مع كلمة المرور المشفرة في قاعدة البيانات.

(6) حفظ بيانات الجلسة

```
$_SESSION['user_id'] = $user['id'];
```

```
$_SESSION['user_name'] = $user['full_name'];
```

```
$_SESSION['user_email'] = $user['email'];
```

هذه البيانات ستستخدم لاحقًا داخل الصفحات المختلفة.

(7) الانتقال إلى الصفحة الرئيسية

```
header("Location: index.php");
```

بعد نجاح الدخول يتم تحويل المستخدم إلى الصفحة الرئيسية.

8) رابط التسجيل

تم انشاء رابط اسفل النموذج يوجه المستخدم الى صفحة التسجيل في حال عدم التسجيل

كود صفحة الخروج

```
<?php
ob_start();
session_start();
$_SESSION = []; // يمسح loggedin, user_id ...
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), "", time() - 42000, $params["path"],
    $params["domain"], $params["secure"], $params["httponly"]);
}
session_destroy();
ob_end_clean();
header("Location: login.php");
exit;
?>
```

شرح الكود

- session_start() يبدأ الجلسة.
- session_destroy() ينهي الجلسة بالكامل.
- ثم يعود المستخدم إلى صفحة الدخول.

كيف نربط تسجيل الدخول بالمشروع؟

بعد نجاح الدخول، سيظهر اسم المستخدم في الأعلى، ويمكن استخدام بيانات الجلسة في:

- لوحة التحكم.
- صفحة الاصناف

• صفحة المصروفات.

وبهذا يصبح كل شيء مرتبطاً بالمستخدم نفسه.

مثال عملي

إذا كتب المستخدم:

• البريد الإلكتروني: user@example.com :

• كلمة المرور 123456 :

وكانت هذه البيانات صحيحة في قاعدة البيانات، سيتم تسجيل دخوله وتحويله إلى الصفحة الرئيسية مباشرة.

أما إذا كانت البيانات خاطئة، فستظهر له رسالة توضح المشكلة.

خلاصة الدرس

في هذا الدرس تعلمنا كيف ننشئ نظام تسجيل الدخول والخروج في مشروع إدارة المصروفات. تعرفنا على صفحة الدخول، وكيف نتحقق من بيانات المستخدم، وكيف نخزن الجلسة، وكيف ننشئ صفحة الخروج.

وبهذا يصبح المشروع أكثر أماناً واحترافية، وأصبح لكل مستخدم بيئة خاصة به.

الدرس الثامن: الصفحة الرئيسية index.php

في هذا الدرس نشرح صفحة الرئيسية للتطبيق، وهي بوابة الدخول إلى النظام بعد تسجيل الدخول، وتعرض هيرو تعريفي وزرًا للانتقال السريع إلى لوحة التحكم.

صفحة index.php

```
<?php
include 'includes/auth.php';
include 'includes/config.php';
include 'includes/header.php';
?>
<section class="hero">
  <div class="container hero-card">
    <div class="hero-grid">
      <div>
        <div class="hero-badges">
          <span class="badge">لوحة تحكم ذكية</span>
          <span class="badge">تقارير ورسوم بيانية</span>
          <span class="badge">دعم العربية</span>
        </div>
        <h2>نظام متكامل لإدارة المصروفات</h2>
        <p>
          تابع ميزانيتك، أضف مصروفاتك بسهولة، اعرض التقارير وراقب الأداء المالي من خلال لوحة تحكم واضحة ورسوم بيانية جذابة صممت لتكون سهلة الاستخدام وسريعة الاستيعاب.
        </p>
        <div style="display:flex; gap:12px; flex-wrap:wrap;">
```

```
<a href="dashboard.php" class="btn btn-primary"> ابدأ من  
</a>  
</div>  
</div>  
<div class="hero-illustration">  
    <div class="bubble">  
        <div class="hero-icon">  
            <svg viewBox="0 0 24 24" aria-hidden="true">  
                <path d="M4 19h16v2H2V3h2v16zm3-2H5v-6h2v6zm5  
0H10V7h2v10zm5 0h-2V4h2v13zm5 0h-2V10h2v7z"/>  
            </svg>  
        </div>  
    </div>  
</div>  
</div>  
</div>  
</div>  
</section>
```

```
<?php include 'includes/footer.php';?>
```

الفكرة العامة للدرس

١. الصفحة الرئيسية هي صفحة ترحيبية بعد الدخول، هدفها تعريف المستخدم بالنظام وتوجيهه للأماكن المهمة بسرعة (لوحة التحكم).
٢. نعيد استخدام ملفات مشتركة (auth.php, config.php, header.php, footer.php) لتقليل تكرار الكود وتحسين تنظيم المشروع.
٣. قسم الهيرو يتكوّن من:
 - شارات صغيرة لعرض أهم مميزات النظام.

- عنوان رئيسي وفقرة تعريفية.
 - زر واضح للانتقال إلى لوحة التحكم.
 - جزء مرئي (أيقونة أو تصميم) يعزز فكرة التطبيق.
٤. استخدام **HTML + CSS** في هذه الصفحة فقط للعرض، ولا يوجد فيها استعلامات قاعدة بيانات خاصة؛ كل منطق البيانات موجود في صفحات أخرى (مثل لوحة التحكم أو صفحة المصروفات).
٥. هذه الصفحة مثال على كيفية تصميم واجهة ترحيبية بسيطة في أي نظام ويب: عنوان واضح، نص مختصر، زر رئيسي، وصورة أو أيقونة معبرة.

الدرس التاسع : إنشاء صفحة التصنيفات

مقدمة الدرس

سنقوم في هذا الدرس بإنشاء **صفحة التصنيفات**. هذه الصفحة مهمة جدًا لأنها تساعدنا على تنظيم المصروفات داخل فئات واضحة مثل الطعام، المواصلات، الدراسة، والفواتير. التصنيفات تجعل البيانات مرتبة وسهلة الفهم، كما أنها تساعد لاحقًا في التقارير والرسوم البيانية والملخصات الشهرية. لذلك نعتبر هذا الدرس من الدروس الأساسية في المشروع.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- ما معنى التصنيفات في نظام إدارة المصروفات.
- كيف ننشئ صفحة لعرض التصنيفات.
- كيف نعرض البيانات القادمة من قاعدة البيانات داخل جدول.
- كيف نضيف أزرار التعديل والحذف لكل تصنيف.
- كيف نربط صفحة التصنيفات بصفحات الإضافة والتعديل والحذف.

ما هي التصنيفات؟

التصنيفات هي أسماء نستخدمها لتنظيم المصروفات حسب نوعها. مثلاً:

- طعام.
- مواصلات.
- دراسة.
- تسوق.
- فواتير.

عندما نسجل أي مصروف، نربطه أولاً بتصنيف معين. فإذا اشترينا وجبة غداء، نختار تصنيف "طعام". وإذا دفعنا أجرة مواصلات، نختار تصنيف "مواصلات".

لماذا نحتاج صفحة التصنيفات؟

نحتاج هذه الصفحة لأن المستخدم يجب أن يرى جميع التصنيفات الموجودة في النظام، ويستطيع:

- إضافة تصنيف جديد.
 - تعديل تصنيف موجود.
 - حذف تصنيف غير مستخدم.
- ومن دون هذه الصفحة ستكون إدارة المصروفات غير منظمة.

اسم الصفحة

عادة نسميها:

categories.php

وهذه الصفحة ستكون المسؤولة عن عرض جميع التصنيفات الموجودة في الجدول **categories**.

كيف تعمل الصفحة؟

عند فتح صفحة التصنيفات، يقوم النظام بالخطوات التالية:

١. الاتصال بقاعدة البيانات.
٢. جلب جميع التصنيفات من جدول **categories**.
٣. عرض التصنيفات داخل جدول **HTML**.
٤. إظهار أزرار الإضافة والتعديل والحذف.

شكل الصفحة المتوقع

الصفحة عادة تحتوي على:

- عنوان الصفحة.
- زر إضافة تصنيف جديد.
- جدول يعرض جميع التصنيفات.
- عمود للإجراءات يحتوي على أزرار التعديل والحذف.

كود صفحة التصنيفات

categories.php: فيما يلي مثال كامل لصفحة

```
<?php include 'includes/auth.php'; ?>
<?php include 'includes/config.php'; ?>
<?php include 'includes/header.php'; ?>

<div class="container" style="margin-top:30px;">
    <div class="card">
        <h2>إدارة التصنيفات</h2>
        <p>من هنا يمكنك عرض جميع التصنيفات وإدارتها بسهولة</p>

        <a href="add_category.php" class="btn btn-primary">إضافة تصنيف جديد</a>

        <table>
            <thead>
                <tr>
                    <th>م</th>
                    <th>اسم التصنيف</th>
                    <th>الإجراءات</th>
                </tr>
            </thead>
            <tbody>
                <?php
                $sql = "SELECT * FROM categories ORDER BY id DESC";
                $result = $conn->query($sql);

                if ($result && $result->num_rows > 0) {
                    $i = 1;
                    while ($row = $result->fetch_assoc()) {
```



```

        echo "<tr>";
        echo "<td>" . $i . "</td>";
        echo "<td>" . htmlspecialchars($row['name']) . "</td>";
        echo "<td>
                <a href='edit_category.php?id=" . $row['id'] . "' class='btn btn-
warning'>تعديل</a>
                <a href='delete_category.php?id=" . $row['id'] . "' class='btn btn-
danger' onclick=\"return confirm('هل أنت متأكد من الحذف؟')\">حذف</a>
        </td>";
        echo "</tr>";
        $i++;
    }
} else {
    echo "<tr><td colspan='3'>لا توجد تصنيفات حتى الآن</td></tr>";
}
?>
</tbody>
</table>
</div>
</div>

<?php include 'includes/footer.php'; ?>

```

شرح الكود

(1) تضمين ملف السيشن والهيدر وملف الاتصال

```

<?php include 'includes/auth.php'; ?>
<?php include 'includes/header.php'; ?>
<?php include 'includes/config.php'; ?>

```

هنا نربط الصفحة بالهيكل العام للمشروع، ونفتح الاتصال بقاعدة البيانات.

2) عنوان الصفحة

```
</h2>إدارة التصنيفات</h2>
```

هذا العنوان يوضح للمستخدم أنه داخل صفحة إدارة التصنيفات.

3) زر إضافة تصنيف

```
<a href="add_category.php" class="btn btn-primary">إضافة تصنيف</a>
```

هذا الزر ينقل المستخدم إلى صفحة إضافة تصنيف جديد.

4) إنشاء جدول HTML

نستخدم جدولاً لعرض التصنيفات بشكل مرتب وواضح.

5) جلب البيانات من قاعدة البيانات

```
$sql = "SELECT * FROM categories ORDER BY id DESC";  
$result = $conn->query($sql);
```

هنا نطلب من قاعدة البيانات إرجاع جميع التصنيفات، مرتبة من الأحدث إلى الأقدم.

6) التحقق من وجود بيانات

```
if ($result && $result->num_rows > 0)
```

إذا كانت هناك بيانات، نعرضها داخل الجدول.
إذا لم توجد بيانات، نعرض رسالة مناسبة.

7) عرض كل تصنيف

```
while ($row = $result->fetch_assoc())
```

نمر على كل سجل من سجلات التصنيفات ونطبعه في صف مستقل داخل الجدول.

8) استخدام htmlspecialchars

```
htmlspecialchars($row['name'])
```

هذا مهم جداً لحماية الصفحة من أي رموز غير مرغوبة ولعرض النصوص بشكل آمن.

9) أزرار التعديل والحذف

كل صف يحتوي على:

• زر تعديل.

• زر حذف.

وهذه الأزرار تنقل المستخدم إلى الصفحات المخصصة لكل عملية.

لماذا نستخدم هذا التصميم؟

لأن المستخدم يحتاج أن يرى التصنيفات في مكان واحد واضح، بدل أن يبحث عنها في أكثر من صفحة.

كما أن وجود جدول منظم يسهل التعديل والحذف والمتابعة.

كيف ترتبط هذه الصفحة بالصفحات الأخرى؟

هذه الصفحة هي الصفحة الرئيسية للتصنيفات، ومنها ننتقل إلى:

• `add_category.php` لإضافة تصنيف.

• `edit_category.php` لتعديل تصنيف.

• `delete_category.php` لحذف تصنيف.

وبهذا يكون لدينا نظام كامل لإدارة التصنيفات.

خلاصة الدرس

في هذا الدرس أنشأنا صفحة التصنيفات التي تعرض جميع التصنيفات الموجودة في قاعدة البيانات داخل جدول منظم. كما أضفنا زرًا لإضافة تصنيف جديد، وأزرارًا للتعديل والحذف. هذه الصفحة تعتبر من أولى الصفحات العملية في المشروع، وهي خطوة مهمة جدًا قبل الانتقال إلى إدارة المصروفات.

الدرس العاشر: إضافة تصنيف جديد

مقدمة الدرس

بعد أن أنشأنا صفحة عرض التصنيفات في الدرس السابق، ننتقل الآن إلى خطوة مهمة جدًا وهي **إضافة تصنيف جديد**. هذه الصفحة تسمح للمستخدم بإنشاء تصنيف جديد مثل: طعام، مواصلات، دراسة، تسوق، أو فواتير. وجود هذه الصفحة ضروري لأن المصروفات في النظام لا بد أن ترتبط بتصنيفات واضحة ومنظمة.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيفية إنشاء صفحة لإضافة تصنيف جديد.
- كيفية بناء نموذج بسيط لإدخال اسم التصنيف.
- كيفية إرسال البيانات إلى قاعدة البيانات.
- كيفية حفظ البيانات بطريقة صحيحة وآمنة.
- كيفية العودة إلى صفحة التصنيفات بعد الإضافة.

فكرة الصفحة

صفحة إضافة التصنيف تكون بسيطة جدًا، لأننا نحتاج فقط إلى حقل واحد وهو **اسم التصنيف**. عندما يكتب المستخدم اسم التصنيف ثم يضغط على زر الحفظ، يتم إرسال البيانات إلى قاعدة البيانات وتخزينها داخل جدول **categories**.

اسم الصفحة

عادة نسميها:

`add_category.php`

ماذا تحتوي الصفحة؟

الصفحة تحتوي على:

- عنوان الصفحة.
- حقل نص لاسم التصنيف.
- زر حفظ.
- مكان لعرض رسائل النجاح أو الخطأ إذا احتجنا ذلك.

```
<?php
session_start();
include 'includes/config.php';

$message = "";

if (isset($_POST['save'])) {
    $name = trim($_POST['name']);

    if (!empty($name)) {
        $stmt = $conn->prepare("INSERT INTO categories (name) VALUES
(?)");
        $stmt->bind_param("s", $name);

        if ($stmt->execute()) {
            header("Location: categories.php");
            exit;
        } else {
            $message = "حدث خطأ أثناء إضافة التصنيف.";
        }

        $stmt->close();
    } else {
        $message = "يرجى إدخال اسم التصنيف.";
    }
}
```

```
?>
```

```
<?php include 'includes/header.php'; ?>
```

```
<div class="container" style="margin-top:30px;">
```

```
  <div class="card">
```

```
    <h2>إضافة تصنيف جديد</h2>
```

```
    <?php if (!empty($message)) : ?>
```

```
      <p style="color:red;"><?php echo $message; ?></p>
```

```
    <?php endif; ?>
```

```
    <form method="post">
```

```
      <label>اسم التصنيف</label>
```

```
      <input type="text" name="name" required>
```

```
      <br><br>
```

```
      <button type="submit" name="save" class="btn btn-  
primary">حفظ التصنيف</button>
```

```
    </form>
```

```
  </div>
```

```
</div>
```

```
<?php include 'includes/footer.php'; ?>
```

```
<?php
```

```

include 'includes/auth.php';
include 'includes/config.php';

$message = "";

if (isset($_POST['save'])) {
    $name = trim($_POST['name']);

    if (!empty($name)) {
        $stmt = $conn->prepare("INSERT INTO categories (name) VALUES
(?)");
        $stmt->bind_param("s", $name);

        if ($stmt->execute()) {
            header("Location: categories.php");
            exit;
        } else {
            $message = "حدث خطأ أثناء إضافة التصنيف";
        }

        $stmt->close();
    } else {
        $message = "يرجى إدخال اسم التصنيف";
    }
}
?>

```

```

<?php include 'includes/header.php'; ?>

<div class="container" style="margin-top:30px;">
    <div class="card" style="max-width:600px; margin:auto;">
        <h2>إضافة تصنيف جديد</h2>

        <?php if (!empty($message)) : ?>
            <p style="color:red;"><?php echo htmlspecialchars($message,
            ENT_QUOTES, 'UTF-8'); ?></p>
        <?php endif; ?>

        <form method="post">
            <label>اسم التصنيف</label>

            <input type="text" name="name" required>

            <br><br>

            <button type="submit" name="save" class="btn btn-
            primary">حفظ التصنيف</button>

        </form>
    </div>
</div>

<?php include 'includes/footer.php'; ?>

```


شرح الكود

(1) بدء الجلسة

`session_start();` تقوم ببدء جلسة جديدة أو استئناف جلسة موجودة حتى تتمكن من تخزين بيانات المستخدم في المصفوفة `$_SESSION` ومشاركتها بين صفحات الموقع.

(1) تضمين ملف الاتصال

```
include 'includes/config.php';
```

نحتاج إلى هذا الملف حتى نستطيع الاتصال بقاعدة البيانات وتنفيذ أمر الحفظ.

(2) إنشاء متغير للرسائل

```
$message = "";
```

نستخدم هذا المتغير لعرض رسالة خطأ إذا حدثت مشكلة أو إذا كان الحقل فارغاً.

(3) التحقق من الضغط على زر الحفظ

```
if (isset($_POST['save'])) {
```

هذا السطر يعني أن المستخدم ضغط على زر الإرسال، وبالتالي تبدأ عملية الحفظ.

(4) قراءة اسم التصنيف

```
$name = trim($_POST['name']);
```

نأخذ الاسم من النموذج ونزيل المسافات الزائدة من البداية والنهاية.

(5) التحقق من أن الحقل غير فارغ

```
if (!empty($name)) {
```

يجب ألا يكون اسم التصنيف فارغاً، لأن التصنيف الفارغ غير مفيد داخل النظام.

(6) استخدام prepared statement

```
$stmt = $conn->prepare("INSERT INTO categories (name) VALUES (?)");
```

```
$stmt->bind_param("s", $name);
```

هذا الأسلوب أكثر أماناً لأنه يمنع بعض المشاكل الأمنية ويجعل إدخال البيانات أكثر تنظيماً.

(7) تنفيذ الحفظ

```
if ($stmt->execute()) {
```

```
header("Location: categories.php");
```

```
exit;
```

}

إذا تم الحفظ بنجاح، نعيد المستخدم إلى صفحة التصنيفات حتى يرى التصنيف الجديد مباشرة.

8) عرض رسالة الخطأ

إذا حدثت مشكلة، تظهر رسالة توضح أن هناك خطأ أثناء الإضافة.

شرح جزء النموذج

```
<form method="post">
  <label>اسم التصنيف</label>
  <input type="text" name="name" required>
  <br><br>
  <button type="submit" name="save" class="btn btn-primary"> حفظ
</button>
</form>
```

ماذا يفعل هذا النموذج؟

- method="post" يعني أن البيانات ستُرسل بشكل مخفي نسبيًا.
- input type="text" هو الحقل الذي يكتب فيه المستخدم اسم التصنيف.
- required تعني أن الحقل يجب ألا يترك فارغًا.
- زر الحفظ يرسل البيانات إلى الخادم.

لماذا نستخدم هذه الطريقة؟

لأنها:

- بسيطة للمبتدئين.
- آمنة أكثر من كتابة الاستعلام مباشرة.
- سهولة التعديل والتوسع لاحقًا.
- تتناسب مع باقي صفحات المشروع.

ملاحظات مهمة للمبتدئين

- تأكدي من كتابة اسم الجدول بشكل صحيح، وهو categories
- لا تتركي اسم التصنيف فارغاً.
- استخدم trim() لإزالة الفراغات الزائدة.
- بعد الحفظ، الأفضل تحويل المستخدم إلى صفحة التصنيفات.
- لا تنسي تضمين ملف الاتصال أولاً.

مثال عملي

إذا كتب المستخدم:

- **طعام**
ثم ضغط زر الحفظ، سيتم إدراج تصنيف جديد باسم "طعام" داخل جدول التصنيفات، وبعدها سيُعاد إلى صفحة التصنيفات ليُشاهد السجل الجديد.

ماذا يحدث بعد هذه الصفحة؟

بعد أن أصبح بإمكاننا إضافة تصنيف جديد، نحتاج إلى صفحة تسمح بتعديل التصنيف إذا كان هناك خطأ في الاسم أو إذا أردنا تغييره لاحقاً. وهذا ما سنشرحه في **الدرس الثامن: تعديل التصنيف**.

خلاصة الدرس

في هذا الدرس تعلمنا كيف ننشئ صفحة لإضافة تصنيف جديد داخل نظام إدارة المصروفات. تعرفنا على فكرة الصفحة، وطريقة كتابة النموذج، وكيفية إرسال البيانات إلى قاعدة البيانات وحفظها بطريقة آمنة. هذه الصفحة مهمة جداً لأنها تمثل الخطوة الأولى في تنظيم المصروفات داخل النظام.

الدرس الحادي عشر: تعديل التصنيف

مقدمة الدرس

بعد أن تعلمنا في الدرس السابق كيف نضيف تصنيفًا جديدًا، ننتقل الآن إلى خطوة مهمة أخرى وهي **تعديل التصنيف**.

أحيانًا يكتب المستخدم اسمًا غير مناسب، أو يريد تغيير اسم التصنيف ليصبح أوضح أو أكثر تنظيمًا. لذلك نحتاج إلى صفحة تسمح له بتعديل الاسم بسهولة.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيفية جلب بيانات التصنيف من قاعدة البيانات.
- كيفية عرض البيانات الحالية داخل النموذج.
- كيفية تعديل اسم التصنيف.
- كيفية حفظ التعديل في قاعدة البيانات.
- كيفية العودة إلى صفحة التصنيفات بعد التحديث.

فكرة الصفحة

صفحة تعديل التصنيف تشبه صفحة الإضافة من حيث الشكل، لكن الفرق الأساسي هو أننا لا نبدأ من الصفر. بل نقوم أولاً بجلب التصنيف المطلوب من قاعدة البيانات، ثم نعرض الاسم الحالي داخل مربع النص، وبعدها يمكن للمستخدم تعديله وحفظ التغيير.

اسم الصفحة

عادة نسميها:

edit_category.php

ماذا تحتوي الصفحة؟

الصفحة تحتوي على:

- عنوان الصفحة.
- حقل اسم التصنيف مع القيمة الحالية.
- زر تحديث.
- رسالة خطأ إذا لم يتم العثور على التصنيف أو إذا حدثت مشكلة.

```
<?php
include 'includes/auth.php';
include 'includes/config.php';

$message = "";

// التأكد من وجود id
if (!isset($_GET['id']) || !is_numeric($_GET['id'])) {
    header("Location: categories.php");
    exit;
}

$id = (int) $_GET['id'];

// جلب بيانات التصنيف الحالي
$stmt = $conn->prepare("SELECT * FROM categories WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$result = $stmt->get_result();
$category = $result->fetch_assoc();
$stmt->close();

if (!$category) {
    header("Location: categories.php");
    exit;
}

// تحديث البيانات عند إرسال النموذج
```

```

if (isset($_POST['update'])) {

    $name = trim($_POST['name']);

    if (!empty($name)) {

        $updateStmt = $conn->prepare("UPDATE categories SET name = ? WHERE id = ?");

        $updateStmt->bind_param("si", $name, $id);

        if ($updateStmt->execute()) {

            header("Location: categories.php");

            exit;

        } else {

            $message = "حدث خطأ أثناء تحديث التصنيف";

        }

        $updateStmt->close();

    } else {

        $message = "يرجى إدخال اسم التصنيف";

    }

}

?>

<?php include 'includes/header.php'; ?>

<div class="container" style="margin-top:30px;">

    <div class="card" style="max-width:600px; margin:auto;">

        <h2>تعديل التصنيف</h2>

        <?php if (!empty($message)) : ?>

```

```

        <p style="color:red;"><?php echo htmlspecialchars($message, ENT_QUOTES,
'UTF-8'); ?></p>

        <?php endif; ?>

        <form method="post">

            <label>اسم التصنيف</label>

            <input type="text" name="name"

                value="<?php echo htmlspecialchars($category['name'], ENT_QUOTES,
'UTF-8'); ?>" required>

            <br><br>

            <button type="submit" name="update" class="btn btn-warning">تحديث
التصنيف</button>

            <a href="categories.php" class="btn btn-secondary">إلغاء</a>

        </form>

    </div>

</div>

<?php include 'includes/footer.php'; ?>

```

شرح الكود

1) تضمين ملف الجلسة و الاتصال

```
include 'includes/auth.php';
```

```
include 'includes/config.php';
```

نحتاج إلى ملف الاتصال حتى نستطيع الاتصال بقاعدة البيانات وجلب بيانات التصنيف.

2) التأكد من وجود رقم التصنيف

```
if (!isset($_GET['id'])) {
```

```
    header("Location: categories.php");
```

```
exit;  
}
```

هذا السطر مهم جداً، لأنه إذا لم يكن هناك رقم للتصنيف المطلوب، فإن الصفحة تعود إلى صفحة التصنيفات.

(3) تحويل الرقم إلى عدد صحيح

```
$id = (int) $_GET['id'];
```

نستخدم هذا السطر لضمان أن قيمة المعرف رقم صحيح وليس نصاً.

(4) جلب التصنيف من قاعدة البيانات

```
$stmt = $conn->prepare("SELECT * FROM categories WHERE id = ?");
```

```
$stmt->bind_param("i", $id);
```

```
$stmt->execute();
```

```
$result = $stmt->get_result();
```

```
$category = $result->fetch_assoc();
```

هنا نبحث عن التصنيف المطلوب باستخدام رقم المعرف، ثم نخزن النتيجة في متغير `$category`.

(5) التحقق من وجود التصنيف

```
if (!$category) {  
    header("Location: categories.php");  
    exit;  
}
```

إذا لم نجد التصنيف، نعيد المستخدم إلى صفحة التصنيفات.

تحديث التصنيف

(1) التحقق من الضغط على زر التحديث

```
if (isset($_POST['update'])) {
```

هذا يعني أن المستخدم أنهى التعديل وضغط على زر التحديث.

2) قراءة الاسم الجديد

```
$name = trim($_POST['name']);
```

نأخذ الاسم الجديد من النموذج ونزيل الفراغات الزائدة.

3) التأكد من أن الاسم غير فارغ

```
if (!empty($name)) {
```

لا يمكن حفظ تصنيف بدون اسم.

4) تنفيذ التحديث

```
$updateStmt = $conn->prepare("UPDATE categories SET name = ?  
WHERE id = ?");
```

```
$updateStmt->bind_param("si", $name, $id);
```

هنا نحدث اسم التصنيف الموجود في قاعدة البيانات.

5) بعد النجاح

```
header("Location: categories.php");
```

```
exit;
```

بعد نجاح التعديل نعود إلى صفحة التصنيفات حتى يرى المستخدم الاسم الجديد.

شرح جزء النموذج

```
<form method="post">
```

```
<label>اسم التصنيف</label>
```

```
<input type="text" name="name" value="<?php echo  
htmlspecialchars($category['name']); ?>" required>
```

```
<br><br>
```

```
<button type="submit" name="update" class="btn btn-  
warning">تحديث التصنيف</button>
```

```
</form>
```

ماذا يفعل هذا النموذج؟

- يعرض الاسم الحالي داخل الحقل.
- يسمح للمستخدم بتعديله.
- يرسل التغيير عند الضغط على زر التحديث.
-

لماذا نستخدم `htmlspecialchars`؟

حتى نعرض الاسم الحالي بشكل آمن، وتجنب ظهور أي رموز غير مرغوبة داخل الصفحة.

لماذا هذا الدرس مهم؟

لأن المستخدم قد يخطئ في كتابة اسم التصنيف عند الإضافة، أو يرغب في تغييره لاحقًا. وجود صفحة تعديل يجعل النظام أكثر مرونة واحترافية.

مثال عملي

إذا كان هناك تصنيف باسم:

- "مواصلات"

وأراد المستخدم تصحيحه إلى:

- "مواصلات"

فيمكنه فتح صفحة التعديل، تغيير الاسم، ثم حفظ التعديل بسهولة.

ملاحظات مهمة للمبتدئين

- تأكدي من أن رقم التصنيف موجود قبل فتح الصفحة.
- لا تنسي إعادة التوجيه إذا كان التصنيف غير موجود.
- استخدمي `trim()` لتنظيف الاسم الجديد.
- بعد التحديث، الأفضل العودة إلى صفحة التصنيفات.
- لا تتركي الحقل فارغًا أثناء التعديل.

خلاصة الدرس

في هذا الدرس تعلمنا كيف نعدل تصنيفًا موجودًا داخل النظام. تعرفنا على طريقة جلب بيانات التصنيف من قاعدة البيانات، وعرضها داخل النموذج، ثم تحديثها وحفظ التغيير. هذه الصفحة مهمة جدًا لأنها تجعل إدارة التصنيفات أسهل وأكثر مرونة.

الدرس الثاني عشر: حذف التصنيف

مقدمة الدرس

بعد أن تعلمنا في الدروس السابقة كيف نعرض التصنيفات ونضيفها ونعدلها، نصل الآن إلى خطوة أخيرة في إدارة التصنيفات وهي **حذف التصنيف**. هذه العملية مهمة عندما يصبح التصنيف غير مستخدم أو لم يعد مناسبًا داخل النظام. لكن يجب تنفيذ الحذف بحذر، لأن حذف التصنيف قد يؤثر على المصروفات المرتبطة به.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيفية حذف تصنيف من قاعدة البيانات.
- كيفية استقبال رقم التصنيف من الرابط.
- كيفية تنفيذ عملية الحذف بشكل آمن.
- كيفية إعادة المستخدم إلى صفحة التصنيفات بعد الحذف.
- كيف نُظهر رسالة تأكيد قبل الحذف.

فكرة الصفحة

صفحة الحذف لا تحتاج إلى نموذج كبير. غالبًا يكفي أن يضغط المستخدم على زر الحذف من صفحة التصنيفات، فينتقل إلى صفحة `delete_category.php`، ثم يتم حذف السجل مباشرة أو بعد التأكيد.

اسم الصفحة

عادة نسميها:

`delete_category.php`

ماذا تفعل هذه الصفحة؟

- تستقبل رقم التصنيف من الرابط.
- تتحقق من وجوده.
- تنفذ عملية الحذف من جدول `categories`.
- تعيد المستخدم إلى صفحة التصنيفات.

```
<?php
include 'includes/auth.php';
include 'includes/config.php';

if (!isset($_GET['id']) || empty($_GET['id'])) {
    header("Location: categories.php");
    exit;
}

$id = (int) $_GET['id'];

$stmt = $conn->prepare("DELETE FROM categories WHERE id = ?");
if (!$stmt) {
    die("Prepare failed: " . $conn->error);
}

$stmt->bind_param("i", $id);

if ($stmt->execute()) {
    header("Location: categories.php");
    exit;
} else {
    die("Execute failed: " . $stmt->error);
}
```

```
$stmt->close();  
?>
```

شرح الكود

1) تضمين السيشن و ملف الاتصال

```
include 'includes/config.php';
```

نحتاج إلى ملف الاتصال بقاعدة البيانات حتى نتمكن من حذف السجل المطلوب.

2) التأكد من وجود رقم التصنيف

```
if (!isset($_GET['id'])) {  
    header("Location: categories.php");  
    exit;  
}
```

إذا لم يكن هناك رقم تصنيف، لا يمكن تنفيذ الحذف، لذلك نعيد المستخدم إلى صفحة التصنيفات.

3) تحويل الرقم إلى عدد صحيح

```
$id = (int) $_GET['id'];
```

هذا يجعل قيمة id آمنة وواضحة كنص عددي.

4) إعداد أمر الحذف

```
$stmt = $conn->prepare("DELETE FROM categories WHERE id = ?");
```

```
$stmt->bind_param("i", $id);
```

هنا نخبر قاعدة البيانات أننا نريد حذف السجل الذي يطابق هذا الرقم.

5) تنفيذ الحذف

```
if ($stmt->execute()) {  
    header("Location: categories.php");  
    exit;
```

}

إذا نجحت العملية، يعود المستخدم إلى صفحة التصنيفات مباشرة.

كيف يتم استدعاء الحذف من صفحة التصنيفات؟

في صفحة categories.php يكون زر الحذف مثل هذا:

```
<a href="delete_category.php?id=<?php echo $row['id']; ?>" class="btn btn-danger" onclick="return confirm('هل أنت متأكد من الحذف?')">حذف</a>
```

ما الذي يحدث هنا؟

- عند الضغط على زر الحذف، يتم إرسال رقم التصنيف إلى صفحة الحذف.
- confirm() تظهر رسالة تأكيد.
- إذا وافق المستخدم، تستمر العملية.
- إذا ألغى، لا يتم الحذف.

لماذا نستخدم رسالة تأكيد؟

لأن الحذف عملية مهمة وقد تؤدي إلى فقدان البيانات. لذلك من الأفضل أن نتأكد من أن المستخدم يريد بالفعل حذف التصنيف قبل تنفيذه.

ملاحظات مهمة جدًا

- لا تحذف التصنيف بدون تأكيد.
- تأكد من أن التصنيف غير مرتبط ببيانات مهمة، خاصة بالمصروفات.
- إذا كان هناك قيود في قاعدة البيانات، قد يفشل الحذف إذا كان التصنيف مستخدمًا في جدول المصروفات.
- في هذه الحالة يمكن التفكير في منع الحذف أو معالجة المصروفات المرتبطة أولاً.

مثال عملي

إذا كان هناك تصنيف باسم "ترفيه" ولم يعد المستخدم يحتاجه، يمكنه الضغط على زر الحذف، ثم التأكيد، وبعدها يُحذف من قاعدة البيانات ويختفي من صفحة التصنيفات.

خلاصة الدرس

في هذا الدرس تعلمنا كيفية حذف التصنيف من قاعدة البيانات بطريقة بسيطة وآمنة. تعرفنا على استقبال رقم التصنيف من الرابط، وتنفيذ أمر الحذف، ثم إعادة المستخدم إلى صفحة التصنيفات. كما تعلمنا أهمية التأكيد قبل الحذف لتجنب فقدان البيانات بالخطأ.

الدرس الثالث عشر: صفحة المصروفات

مقدمة الدرس

بعد أن أنشأنا صفحة التصنيفات وأصبح بإمكاننا إدارتها بشكل كامل، ننتقل الآن إلى الجزء الأهم في المشروع وهو **صفحة المصروفات**. هذه الصفحة هي القلب الحقيقي لنظام إدارة المصروفات، لأنها المسؤولة عن تسجيل كل عملية صرف يقوم بها المستخدم. من خلالها نستطيع معرفة ماذا تم صرفه، ومتى تم صرفه، ولأي تصنيف كان المصروف.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- ما المقصود بالمصروفات داخل النظام.
- كيف ننشئ صفحة لعرض المصروفات.
- كيف نربط كل مصروف بتصنيف معين.
- كيف نعرض بيانات المصروفات داخل جدول.
- كيف نجهز الصفحة لتصبح قابلة للإضافة والتعديل والحذف لاحقاً.

ما هي المصروفات؟

المصروف هو أي مبلغ يخرج من المستخدم من ماله، مثل:

- شراء طعام.
- دفع أجرة مواصلات.
- شراء أدوات دراسية.
- دفع فاتورة.
- أي تكلفة أخرى.

كل مصروف يتم تسجيله داخل النظام حتى يمكن الرجوع إليه لاحقاً وتحليله ومقارنته بالميزانية.

لماذا صفحة المصروفات مهمة؟

هذه الصفحة مهمة جداً لأنها تمثل البيانات الأساسية التي سيبني عليها المشروع باقي الوظائف مثل:

- التقارير.
- الملخص الشهري.

- الرسوم البيانية.
- الميزانية.
- التنبيهات.

بمعنى آخر، بدون المصروفات لن يكون للنظام قيمة حقيقية.

اسم الصفحة

عادة نسمي الصفحة الرئيسية للمصروفات:

expenses.php

ماذا تعرض الصفحة؟

صفحة المصروفات غالبًا تعرض:

- رقم المصروف.
- اسم التصنيف.
- قيمة المصروف.
- الوصف.
- تاريخ المصروف.
- أزرار التعديل والحذف.
- زر لإضافة مصروف جديد.

كود صفحة عرض المصروفات

فيما يلي مثال كامل لصفحة expenses.php:

```
<?php include 'includes/auth.php'; ?>
<?php include 'includes/config.php'; ?>
<?php include 'includes/header.php'; ?>

<div class="container" style="margin-top:30px;">
  <div class="card">
```

<h2>إدارة المصروفات</h2>

<p>من هنا يمكنك عرض جميع المصروفات المسجلة في النظام</p>

إضافة مصروف جديد

<table>

<thead>

<tr>

<th>م</th>

<th>التصنيف</th>

<th>المبلغ</th>

<th>الوصف</th>

<th>التاريخ</th>

<th>الإجراءات</th>

</tr>

</thead>

<tbody>

<?php

\$sql = "SELECT expenses.id, categories.name AS
category_name, expenses.amount, expenses.description,
expenses.expense_date

FROM expenses

INNER JOIN categories ON expenses.category_id =
categories.id

ORDER BY expenses.id DESC";

```

$result = $conn->query($sql);

if ($result && $result->num_rows > 0) {
    $i = 1;
    while ($row = $result->fetch_assoc()) {
        echo "<tr>";
        echo "<td>" . $i . "</td>";
        echo "<td>" . htmlspecialchars($row['category_name']) .
"</td>";
        echo "<td>" . number_format($row['amount'], 2) . "</td>";
        echo "<td>" . htmlspecialchars($row['description']) .
"</td>";
        echo "<td>" . htmlspecialchars($row['expense_date']) .
"</td>";
        echo "<td>
            <a href='edit_expense.php?id=" . $row['id'] . "'
class='btn btn-warning'>تعديل</a>
            <a href='delete_expense.php?id=" . $row['id'] . "'
class='btn btn-danger' onclick=\"return confirm('
هل أنت متأكد من
الحذف؟')\">حذف</a>
        </td>";
        echo "</tr>";
        $i++;
    }
} else {
    echo "<tr><td colspan='6'>لا توجد مصروفات حتى الآن</td></tr>";
}
?>

```

```

        </tbody>
    </table>
</div>
</div>

<?php include 'includes/footer.php'; ?>

```

شرح الكود

1) تضمين السيشن ، الهيدر وملف الاتصال

```

<?php include 'includes/session.php'; ?>
<?php include 'includes/header.php'; ?>
<?php include 'includes/config.php'; ?>

```

الهيدر يضيف الشكل العام للموقع، وملف الاتصال يربط الصفحة بقاعدة البيانات.

2) عنوان الصفحة

```
<h2>إدارة المصروفات</h2>
```

هذا العنوان يوضح أن الصفحة مخصصة لإدارة المصروفات.

3) زر إضافة مصروف جديد

```

<a href="add_expense.php" class="btn btn-primary">إضافة مصروف
جديد</a>

```

هذا الزر ينقل المستخدم إلى صفحة إضافة مصروف.

4) إنشاء جدول العرض

نستخدم جدول HTML لعرض المصروفات بشكل منظم وواضح.

5) جلب البيانات من قاعدة البيانات

php

```

$sql = "SELECT expenses.id, categories.name AS category_name,
expenses.amount, expenses.description, expenses.expense_date

```

FROM expenses

INNER JOIN categories ON expenses.category_id = categories.id

ORDER BY expenses.id DESC";

هذا الاستعلام مهم جدًا لأنه لا يعرض فقط المصروفات، بل يجلب أيضًا اسم التصنيف من جدول categories.

6) استخدام INNER JOIN

نستخدم الربط بين الجدولين لأن المصروف الواحد مرتبط بتصنيف واحد. وبالتالي نستطيع عرض اسم التصنيف بدل رقم category_id.

7) التحقق من وجود بيانات

```
if ($result && $result->num_rows > 0)
```

إذا كانت هناك مصروفات، نعرضها داخل الجدول.
إذا لم تكن هناك بيانات، نعرض رسالة مناسبة.

8) عرض كل مصروف

```
while ($row = $result->fetch_assoc())
```

نمر على كل سجل من سجلات المصروفات ونضيفه في صف جديد.

9) تنسيق المبلغ

```
number_format($row['amount'], 2)
```

هذا السطر يجعل المبلغ ظاهرًا بشكل منظم مع منزلتين عشريتين.

10) أزرار التعديل والحذف

كل مصروف يحتوي على:

- زر تعديل.

- زر حذف.

وهذا يجهز الصفحة للخطوات القادمة.

لماذا نستخدم الربط بين expenses و categories؟

لأن المصروف يُخزن في جدول expenses برقم التصنيف فقط، لكن المستخدم يحتاج أن يرى اسم التصنيف.

لذلك نستخدم INNER JOIN حتى نعرض اسم التصنيف الحقيقي بدل الرقم.

كيف تعمل الصفحة من الناحية العملية؟

عندما يفتح المستخدم صفحة المصروفات:

١. يتم الاتصال بقاعدة البيانات.
٢. يتم تنفيذ الاستعلام.
٣. يتم جلب المصروفات مع أسماء تصنيفاتها.
٤. يتم عرض النتائج في جدول.
٥. يمكن للمستخدم الانتقال للإضافة أو التعديل أو الحذف.

ملاحظات مهمة للمبتدئين

- تأكدي من أن جدول expenses مرتبط بجدول categories.
- لا تنسي استخدام htmlspecialchars عند عرض النصوص.
- استخدم number_format لتنسيق المبالغ.
- احرصي على أن تكون أسماء الحقول صحيحة تمامًا.
- إذا لم توجد مصروفات، أظهري رسالة بديلة للمستخدم.

مثال عملي

إذا كان هناك مصروف بقيمة ٣٥,٠٠ ريال لتصنيف "طعام"، فستظهر في الجدول هكذا:

- التصنيف: طعام
- المبلغ: ٣٥,٠٠
- الوصف: غداء
- التاريخ: ٢٦-٠٤-٢٠٢٦

لماذا هذا الدرس مهم؟

لأن هذه الصفحة هي الواجهة الأساسية لمتابعة المصروفات. ومن خلالها نستطيع لاحقًا إضافة سجلات جديدة، تعديل البيانات، وحذف أي مصروف غير صحيح.

خلاصة الدرس

في هذا الدرس تعلمنا كيفية إنشاء صفحة عرض المصروفات. تعرفنا على طريقة جلب البيانات من جدول expenses وربطها بجدول categories حتى نعرض اسم التصنيف بدل رقمه. كما تعلمنا كيف ننظم المصروفات داخل جدول HTML مع أزرار التعديل والحذف.

الدرس الربع عشر: إضافة مصروف جديد

مقدمة الدرس

بعد أن تعلمنا في الدرس السابق كيف نعرض المصروفات داخل جدول منظم، ننتقل الآن إلى الجزء الأهم عملياً، وهو **إضافة مصروف جديد**. هذه الصفحة تسمح للمستخدم بتسجيل أي عملية صرف يقوم بها، مثل شراء طعام أو دفع مواصلات أو شراء مستلزمات دراسية. كل مصروف يتم حفظه في قاعدة البيانات حتى نستطيع الرجوع إليه لاحقاً في التقارير والملخصات.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيفية إنشاء صفحة لإضافة مصروف جديد.
- كيفية اختيار التصنيف المناسب للمصروف.
- كيفية إدخال المبلغ والوصف والتاريخ.
- كيفية حفظ المصروف في قاعدة البيانات.
- كيفية العودة إلى صفحة المصروفات بعد الإضافة.

فكرة الصفحة

صفحة إضافة المصروف تشبه صفحة إضافة التصنيف، لكنها تحتوي على عدة حقول بدلاً من حقل واحد فقط. نحتاج عادة إلى:

- اختيار التصنيف.
- إدخال المبلغ.
- كتابة وصف بسيط.
- تحديد تاريخ المصروف.

بهذه الطريقة يصبح كل مصروف محفوظاً بشكل واضح ومفهوم.

اسم الصفحة

عادة نسميها:

add_expense.php

ماذا تحتوي الصفحة؟

الصفحة تحتوي على:

- عنوان الصفحة.
- قائمة منسدلة للتصنيفات.
- حقل للمبلغ.
- حقل للوصف.
- حقل للتاريخ.
- زر حفظ.
- رسالة خطأ إذا كانت هناك مشكلة في الإدخال.

كود صفحة إضافة المصروف

```
<?php
include 'includes/auth.php';
include 'includes/config.php';

$message = "";

// جلب التصنيفات
$categories = [];
$result = $conn->query("SELECT id, name FROM categories ORDER BY name ASC");
if ($result && $result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        $categories[] = $row;
    }
}

if (isset($_POST['save'])) {
    $category_id = (int) $_POST['category_id'];
```

```

$amount = (float) $_POST['amount'];
$description = trim($_POST['description']);
$expense_date = trim($_POST['expense_date']);

if ($category_id > 0 && $amount > 0 && !empty($expense_date)) {
    $user_id = (int) $_SESSION['user_id'];

    $stmt = $conn->prepare("INSERT INTO expenses (user_id, category_id, amount,
description, expense_date) VALUES (?, ?, ?, ?, ?)");

    if (!$stmt) {
        die("Prepare failed: " . $conn->error);
    }

    $stmt->bind_param("iidss", $user_id, $category_id, $amount, $description,
$expense_date);

    if ($stmt->execute()) {
        header("Location: expenses.php");
        exit;
    } else {
        $message = "حدث خطأ أثناء إضافة المصروف: " . $stmt->error;
    }

    $stmt->close();
} else {
    $message = "يرجى تعبئة الحقول المطلوبة";
}
}

include 'includes/header.php';

```

?>

```
<div class="container" style="margin-top:30px;">

  <div class="card" style="max-width:600px; margin:auto;">

    <h2>إضافة مصروف جديد</h2>

    <?php if (!empty($message)) : ?>
      <p style="color:red;"><?php echo htmlspecialchars($message); ?></p>
    <?php endif; ?>

    <form method="post">
      <label>التصنيف</label>
      <select name="category_id" required>
        <option value="">اختر التصنيف</option>
        <?php foreach ($categories as $category) : ?>
          <option value="<?php echo $category['id']; ?>">
            <?php echo htmlspecialchars($category['name']); ?>
          </option>
        <?php endforeach; ?>
      </select>

      <br><br>

      <label>المبلغ</label>
      <input type="number" name="amount" step="0.01" required>

      <br><br>

      <label>الوصف</label>
```

```

<input type="text" name="description">

<br><br>

<label>تاريخ المصروف</label>
<input type="date" name="expense_date" required>

<br><br>

<button type="submit" name="save" class="btn btn-primary"> حفظ
المصروف</button>
</form>
</div>
</div>

<?php include 'includes/footer.php'; ?>

```

شرح الكود

1) تضمين ملف الاتصال

```
include 'includes/config.php';
```

نحتاج إلى هذا الملف حتى نستطيع الوصول إلى قاعدة البيانات.

2) إنشاء متغير للرسائل

```
$message = "";
```

نستخدمه لعرض أي رسالة خطأ أو تنبيه للمستخدم.

3) جلب التصنيفات

```
$result = $conn->query("SELECT id, name FROM categories ORDER BY
name ASC");
```

نحتاج إلى التصنيفات حتى نعرضها في القائمة المنسدلة، كي يختار المستخدم نوع المصروف.

4) تخزين التصنيفات داخل مصفوفة

```
while ($row = $result->fetch_assoc()) {  
    $categories[] = $row;  
}
```

هنا نحول نتائج قاعدة البيانات إلى مصفوفة يمكن استخدامها داخل صفحة HTML.

5) التحقق من الضغط على زر الحفظ

```
if (isset($_POST['save'])) {
```

هذا السطر يعني أن المستخدم أرسل النموذج ويريد حفظ المصروف.

6) قراءة البيانات

```
$category_id = (int) $_POST['category_id'];  
$amount = trim($_POST['amount']);  
$description = trim($_POST['description']);  
$expense_date = trim($_POST['expense_date']);
```

نأخذ القيم من النموذج:

- التصنيف.
- المبلغ.
- الوصف.
- التاريخ.

7) التحقق من الحقول المطلوبة

```
if (!empty($category_id) && !empty($amount) &&  
    !empty($expense_date)) {
```

بعض الحقول مهمة جدًا ولا يمكن تركها فارغة.

8) تجهيز أمر الإضافة

```
$stmt = $conn->prepare("INSERT INTO expenses (user_id, category_id,  
amount, description, expense_date) VALUES (?, ?, ?, ?, ?)");
```

expenses. هذا الأمر يضيف المصروف إلى جدول

9) ربط القيم

في الأصل يجب ربط القيم بالترتيب الصحيح، لكن بسبب بساطة الشرح هنا نفهم أن:

- user_id يمثل المستخدم الحالي.
- category_id يمثل التصنيف.
- amount يمثل قيمة المصروف.
- description يمثل وصفًا اختياريًا.
- expense_date يمثل تاريخ المصروف.

10) بعد النجاح

```
header("Location: expenses.php");
```

```
exit;
```

بعد حفظ المصروف نعود إلى صفحة المصروفات.

شرح النموذج

اختيار التصنيف

```
<select name="category_id" required>
```

هذا الحقل يعرض جميع التصنيفات المسجلة في النظام.

إدخال المبلغ

```
<input type="number" name="amount" step="0.01" required>
```

نستخدم number حتى يسمح للمستخدم بإدخال قيمة رقمية فقط.

إدخال الوصف

```
<input type="text" name="description">
```

هذا الحقل اختياري، ويمكن كتابة وصف قصير للمصروف.

إدخال التاريخ

```
<input type="date" name="expense_date" required>
```

هذا الحقل يحدد متى حدث المصروف.

ملاحظات مهمة جدًا

- يجب التأكد من أن التصنيف موجود قبل الإضافة.
- يجب أن يكون المبلغ رقمًا صحيحًا أو عشريًا.
- الأفضل ربط المصروف بالمستخدم الحالي من خلال الجلسة لاحقًا.
- لا تترك الحقول المهمة فارغة.
- استخدمي `htmlspecialchars` عند عرض أسماء التصنيفات.

مثال عملي

إذا أراد المستخدم تسجيل مصروف غداء، يمكنه:

- اختيار التصنيف: طعام.
- إدخال المبلغ: ٢٥.
- كتابة الوصف: غداء في الخارج.
- اختيار التاريخ: اليوم.
- ثم الضغط على حفظ.

بعدها يتم تسجيل المصروف في قاعدة البيانات ويُعاد إلى صفحة المصروفات.

لماذا هذا الدرس مهم؟

لأن صفحة إضافة المصروف هي الصفحة التي يبدأ منها إدخال البيانات الحقيقية في النظام. ومن دونها لن يكون هناك أي سجل للمصروفات يمكن عرضه أو تحليله لاحقًا.

خلاصة الدرس

في هذا الدرس تعلمنا كيف ننشئ صفحة لإضافة مصروف جديد. تعرفنا على عناصر النموذج، وكيف نعرض التصنيفات داخل قائمة منسدلة، وكيف نأخذ المبلغ والوصف والتاريخ، ثم نحفظ المصروف في قاعدة البيانات. هذه الصفحة هي واحدة من أهم صفحات المشروع لأنها تمثل النقطة التي تبدأ منها بيانات النظام الحقيقي.

الدرس الخامس عشر: تعديل المصروف

مقدمة الدرس

بعد أن تعلمنا في الدرس السابق كيف نضيف مصروفًا جديدًا، ننتقل الآن إلى خطوة مهمة جدًا وهي **تعديل المصروف**. أحيانًا يدخل المستخدم بيانات غير صحيحة مثل مبلغ خاطئ أو تاريخ غير مناسب أو تصنيف غير صحيح، لذلك نحتاج إلى صفحة تسمح له بتعديل المصروف بسهولة.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيفية جلب بيانات المصروف من قاعدة البيانات.
- كيفية عرض البيانات الحالية داخل النموذج.
- كيفية تعديل المصروف وحفظ التغييرات.
- كيفية العودة إلى صفحة المصروفات بعد التحديث.
- كيفية التعامل مع رقم المصروف القادم من الرابط.

فكرة الصفحة

صفحة تعديل المصروف تشبه صفحة الإضافة، ولكن الفرق هنا أننا لا نبدأ من الصفر. نقوم أولاً بجلب بيانات المصروف المطلوب من قاعدة البيانات، ثم نعرضها داخل النموذج. بعدها يستطيع المستخدم تعديل:

- التصنيف.
- المبلغ.
- الوصف.
- التاريخ.

اسم الصفحة

عادة نسميها:

edit_expense.php

ماذا تحتوي الصفحة؟

الصفحة تحتوي على:

- عنوان الصفحة.
- نموذج يعرض البيانات الحالية.
- قائمة منسدلة للتصنيفات.
- حقل المبلغ.
- حقل الوصف.
- حقل التاريخ.
- زر تحديث.
- رسائل خطأ إذا احتجنا.

كود صفحة تعديل المصروف

```
<?php
include 'includes/auth.php';
include 'includes/config.php';

$user_id = (int) $_SESSION['user_id'];

if (!isset($_GET['id'])) {
    header("Location: expenses.php");
    exit;
}

$id = (int) $_GET['id'];
$message = "";

// جلب جميع التصنيفات
$categories = [];
```

```

$catResult = $conn->query("SELECT id, name FROM categories ORDER BY name
ASC");

if ($catResult && $catResult->num_rows > 0) {
    while ($row = $catResult->fetch_assoc()) {
        $categories[] = $row;
    }
}

// جلب بيانات المصروف الحالي للمستخدم الحالي فقط
$stmt = $conn->prepare("SELECT * FROM expenses WHERE id = ? AND user_id = ?");
$stmt->bind_param("ii", $id, $user_id);
$stmt->execute();
$result = $stmt->get_result();
$expense = $result->fetch_assoc();
$stmt->close();

if (!$expense) {
    header("Location: expenses.php");
    exit;
}

if (isset($_POST['update'])) {
    $category_id = (int) $_POST['category_id'];
    $amount = (float) $_POST['amount'];
    $description = trim($_POST['description']);
    $expense_date = trim($_POST['expense_date']);

    if ($category_id > 0 && $amount > 0 && !empty($expense_date)) {
        $updateStmt = $conn->prepare("UPDATE expenses SET category_id = ?, amount
= ?, description = ?, expense_date = ? WHERE id = ? AND user_id = ?");
    }
}

```

```

$updateStmt->bind_param("idssii", $category_id, $amount, $description,
$expense_date, $id, $user_id);

if ($updateStmt->execute()) {
    header("Location: expenses.php");
    exit;
} else {
    $message = "حدث خطأ أثناء تحديث المصروف";
}

$updateStmt->close();
} else {
    $message = "يرجى تعبئة الحقول المطلوبة";
}
}
?>

<?php include 'includes/header.php'; ?>

<div class="container" style="margin-top:30px;">
    <div class="card" style="max-width:600px; margin:auto;">
        <h2>تعديل المصروف</h2>

        <?php if (!empty($message)) : ?>
            <p style="color:red;"><?php echo htmlspecialchars($message, ENT_QUOTES,
'UTF-8'); ?></p>
        <?php endif; ?>

        <form method="post">
            <label>التصنيف</label>

```

```

<select name="category_id" required>
    <?php foreach ($categories as $category) : ?>
        <option value="<?php echo $category['id']; ?>" <?php if ($category['id']
== $expense['category_id']) echo 'selected'; ?>>
            <?php echo htmlspecialchars($category['name'], ENT_QUOTES, 'UTF-
8'); ?>
        </option>
    <?php endforeach; ?>
</select>

<br><br>

<label>المبلغ</label>

<input type="number" name="amount" step="0.01" value="<?php echo
htmlspecialchars($expense['amount'], ENT_QUOTES, 'UTF-8'); ?>" required>

<br><br>

<label>الوصف</label>

<input type="text" name="description" value="<?php echo
htmlspecialchars($expense['description'], ENT_QUOTES, 'UTF-8'); ?>">

<br><br>

<label>تاريخ المصروف</label>

<input type="date" name="expense_date" value="<?php echo
htmlspecialchars($expense['expense_date'], ENT_QUOTES, 'UTF-8'); ?>" required>

<br><br>

```

```
<button type="submit" name="update" class="btn btn-warning">تحديث</button>المصروف</form></div></div><?php include 'includes/footer.php'; ?>
```

شرح الكود

1) تضمين ملف الاتصال

```
include 'includes/config.php';
```

نحتاج إلى قاعدة البيانات حتى نتمكن من جلب المصروف وتحديثه.

2) التأكد من وجود رقم المصروف

```
if (!isset($_GET['id'])) {  
    header("Location: expenses.php");  
    exit;  
}
```

إذا لم يأت رقم المصروف من الرابط، فلا يمكن تعديل أي شيء، لذلك نرجع إلى صفحة المصروفات.

3) تحويل الرقم إلى عدد صحيح

```
$id = (int) $_GET['id'];
```

هذا يجعل قيمة المعرف آمنة وواضحة.

4) جلب التصنيفات

نحتاج إلى جميع التصنيفات حتى نعرضها في القائمة المنسدلة.

5) جلب بيانات المصروف الحالي

```
$stmt = $conn->prepare("SELECT * FROM expenses WHERE id = ?");
```

هذا الاستعلام يجلب المصروف المطلوب بناءً على رقمه.

(6) التحقق من وجود المصروف

```
if (!$expense) {  
    header("Location: expenses.php");  
    exit;  
}
```

إذا لم نجد المصروف، نعيد المستخدم إلى صفحة المصروفات.

تحديث المصروف

(1) التحقق من الضغط على زر التحديث

```
if (isset($_POST['update'])) {
```

هذا يعني أن المستخدم عدل البيانات ويريد حفظها.

(2) قراءة القيم الجديدة

```
$category_id = (int) $_POST['category_id'];  
$amount = trim($_POST['amount']);  
$description = trim($_POST['description']);  
$expense_date = trim($_POST['expense_date']);
```

نأخذ البيانات الجديدة من النموذج.

(3) التحقق من الحقول المطلوبة

```
if (!empty($category_id) && !empty($amount) &&  
    !empty($expense_date)) {
```

لا يمكن حفظ مصروف بدون هذه القيم الأساسية.

(4) تنفيذ التحديث

```
$updateStmt = $conn->prepare("UPDATE expenses SET category_id = ?,  
amount = ?, description = ?, expense_date = ? WHERE id = ?");
```

هنا نحدث السجل الموجود في قاعدة البيانات.

5) بعد النجاح

```
header("Location: expenses.php");
```

```
exit;
```

بعد التحديث نعود إلى صفحة المصروفات.

شرح النموذج

قائمة التصنيفات

```
<select name="category_id" required>
```

تعرض جميع التصنيفات الموجودة في النظام.

تحديد التصنيف الحالي

```
<?php if ($category['id'] == $expense['category_id']) echo 'selected'; ?>
```

هذا السطر يجعل التصنيف الحالي ظاهرًا كمحدد افتراضي داخل القائمة.

حقل المبلغ

```
<input type="number" name="amount" step="0.01" value="<?php echo  
htmlspecialchars($expense['amount']); ?>" required>
```

يعرض المبلغ الحالي ويمكن تعديله.

حقل الوصف

```
<input type="text" name="description" value="<?php echo  
htmlspecialchars($expense['description']); ?>">
```

يعرض الوصف الحالي داخل الحقل.

حقل التاريخ

```
<input type="date" name="expense_date" value="<?php echo  
htmlspecialchars($expense['expense_date']); ?>" required>
```

يعرض تاريخ المصروف الحالي.

لماذا هذا الدرس مهم؟

لأن المستخدم قد يكتشف بعد الإضافة أن هناك خطأ في:

- المبلغ.
- التصنيف.
- الوصف.
- التاريخ.

لذلك وجود صفحة تعديل يجعل النظام مرناً وسهل الاستخدام.

مثال عملي

إذا كان هناك مصروف بقيمة ٥٠ ريال تحت تصنيف "مواصلات"، ثم اكتشف المستخدم أن القيمة الصحيحة هي ٣٥ ريال فقط، فيمكنه فتح صفحة التعديل، تعديل المبلغ، ثم حفظ التحديث.

ملاحظات مهمة للمبتدئين

- تأكدي من وجود رقم المصروف قبل فتح الصفحة.
- لا تنسي جلب المصروف من قاعدة البيانات قبل عرض النموذج.
- استخدم `selected` داخل القائمة المنسدلة.
- استخدم `htmlspecialchars` عند عرض القيم.
- بعد الحفظ، أعيدي المستخدم إلى صفحة المصروفات.

خلاصة الدرس

في هذا الدرس تعلمنا كيف نعدل مصروفًا موجودًا داخل النظام. تعرفنا على طريقة جلب بيانات المصروف، وعرضها داخل النموذج، ثم تعديلها وحفظ التغييرات في قاعدة البيانات. هذه الصفحة مهمة جدًا لأنها تمنح المستخدم مرونة كاملة في إدارة بياناته.

الدرس السادس عشر: حذف المصروف

مقدمة الدرس

بعد أن تعلمنا في الدرس السابق كيف نعدل المصروف، ننتقل الآن إلى خطوة مهمة جدًا وهي **حذف المصروف**. هذه العملية يحتاجها المستخدم عندما يسجل مصروفًا بالخطأ، أو عندما يريد إزالة سجل لم يعد صحيحًا أو لم يعد بحاجة إليه. لكن الحذف يجب أن يتم بحذر، لأن المصروف المحذوف لن يظهر مرة أخرى في التقارير أو الجداول.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيفية حذف المصروف من قاعدة البيانات.
- كيفية استلام رقم المصروف من الرابط.
- كيفية تنفيذ الحذف بشكل آمن.
- كيفية إعادة المستخدم إلى صفحة المصروفات بعد الحذف.
- كيفية إضافة رسالة تأكيد قبل الحذف.

فكرة الصفحة

صفحة حذف المصروف تكون بسيطة جدًا، لأننا لا نحتاج إلى نموذج كامل. كل ما نحتاجه هو:

- زر حذف من صفحة المصروفات.
- صفحة تستقبل رقم المصروف.
- أمر حذف من قاعدة البيانات.
- إعادة توجيهه إلى صفحة المصروفات بعد التنفيذ.

اسم الصفحة

عادة نسميها:

delete_expense.php

ماذا تفعل هذه الصفحة؟

- تستقبل رقم المصروف من الرابط.
- تتحقق من وجوده.
- تحذف السجل من جدول expenses.
- تعيد المستخدم إلى صفحة المصروفات.

كود صفحة حذف المصروف

```
<?php
include 'includes/auth.php';
include 'includes/config.php';

if (!isset($_GET['id'])) {
    header("Location: expenses.php");
    exit;
}

$id = (int) $_GET['id'];

$stmt = $conn->prepare("DELETE FROM expenses WHERE id = ?");
$stmt->bind_param("i", $id);

if ($stmt->execute()) {
    header("Location: expenses.php");
    exit;
} else {
    echo "حدث خطأ أثناء حذف المصروف";
}

$stmt->close();
?>
```

شرح الكود

1) تضمين ملف الاتصال

```
include 'includes/config.php';
```

نحتاج إلى هذا الملف حتى نستطيع الاتصال بقاعدة البيانات وتنفيذ أمر الحذف.

2) التأكد من وجود رقم المصروف

```
if (!isset($_GET['id'])) {  
    header("Location: expenses.php");  
    exit;  
}
```

إذا لم يتم إرسال رقم المصروف، فلا يمكن حذف أي سجل، لذلك نعيد المستخدم إلى صفحة المصروفات.

3) تحويل الرقم إلى عدد صحيح

```
$id = (int) $_GET['id'];
```

هذا يجعل قيمة id رقمية وواضحة.

4) إعداد أمر الحذف

```
$stmt = $conn->prepare("DELETE FROM expenses WHERE id = ?");  
$stmt->bind_param("i", $id);
```

هنا نخبر قاعدة البيانات أننا نريد حذف السجل الذي يحمل هذا الرقم.

5) تنفيذ الحذف

```
if ($stmt->execute()) {  
    header("Location: expenses.php");  
    exit;  
}
```

إذا نجحت العملية، يتم تحويل المستخدم مباشرة إلى صفحة المصروفات.

كيف نضيف زر الحذف في صفحة المصروفات؟

في صفحة expenses.php نضع زر الحذف بجانب كل مصروف مثل هذا:

```
<a href="delete_expense.php?id=?php echo $row['id']; ?>" class="btn btn-danger" onclick="return confirm('هل أنت متأكد من حذف ')>حذف</a>
```

ماذا يحدث هنا؟

- عند الضغط على الزر، يتم إرسال رقم المصروف إلى صفحة الحذف.
- تظهر رسالة تأكيد قبل التنفيذ.
- إذا وافق المستخدم، يتم الحذف.
- إذا ألغى، لا يحدث شيء.

لماذا نستخدم رسالة تأكيد؟

لأن الحذف عملية حساسة وقد تؤدي إلى فقدان بيانات مهمة. لذلك من الأفضل أن نطلب من المستخدم التأكد أولاً قبل تنفيذ الحذف.

مثال عملي

إذا سجل المستخدم مصروفًا بالخطأ بقيمة ١٠٠ ريال بدل ١٠ ريال، فيمكنه:

١. الضغط على زر الحذف.
٢. تأكيد العملية.
٣. حذف السجل الخاطئ.
٤. ثم إضافة المصروف الصحيح من جديد.

لماذا هذا الدرس مهم؟

لأن إدارة البيانات لا تكتمل إلا بثلاث عمليات أساسية:

- الإضافة.
- التعديل.
- الحذف.

وبذلك يصبح لدينا نظام CRUD كامل للمصروفات.

ملاحظات مهمة للمبتدئين

- لا تحذف المصروف بدون رسالة تأكيد.
- تأكدي من وجود رقم المصروف قبل الحذف.
- بعد الحذف، أعيدي المستخدم إلى صفحة المصروفات.
- استخدمي الحذف بحذر لأن البيانات قد لا يمكن استرجاعها بسهولة.
- إذا كان المصروف مرتبطًا بالتقارير، فالحذف سيؤثر على النتائج لاحقًا.

خلاصة الدرس

في هذا الدرس تعلمنا كيفية حذف المصروف من قاعدة البيانات بطريقة بسيطة وآمنة. تعرفنا على استقبال رقم المصروف من الرابط، وتنفيذ أمر الحذف، ثم إعادة المستخدم إلى صفحة المصروفات. كما تعلمنا أهمية رسالة التأكيد قبل الحذف لتجنب فقدان البيانات عن طريق الخطأ.

الدرس السابع عشر: صفحة الميزانية

1. مقدمة الدرس

في هذا الدرس نتعرّف على صفحة الميزانية الشهرية في نظام إدارة المصروفات، وهي الصفحة التي تسمح للمستخدم بإدخال ميزانية لكل شهر وسنة، وتعديلها عند الحاجة. هذه الميزانية تستخدم لاحقاً في لوحة التحكم لحساب نسبة الصرف ومقارنة المصروفات بالحد المسموح.

2. هدف الدرس

- أن تفهم الطالبة كيف تربط بين النموذج (Form) وقاعدة البيانات.
- أن تتعلم طريقة التحقق: إذا كانت الميزانية موجودة يتم تحديثها، وإذا لم تكن موجودة يتم إضافتها.
- أن تطبق استخدام Prepared Statements مع SELECT وINSERT وUPDATE.
- أن ترى مثلاً على تعبئة النموذج بقيم افتراضية بناءً على بيانات محفوظة مسبقاً.

3. أهمية هذا الدرس في المشروع

- بدون صفحة الميزانية لن يعرف النظام ما هو الحد الشهري للإنفاق لكل مستخدم، وبالتالي لا يمكنه إظهار حالة الميزانية (مستقرة، قريبة من التجاوز، متجاوزة).
- هذا الدرس يربط بين:
 - جدول budgets في قاعدة البيانات،
 - صفحة لوحة التحكم التي تعرض نسبة الصرف،
 - وتجربة المستخدم الذي يحدد بنفسه ميزانية كل شهر.
- المفهوم المستخدم هنا (تحديث إذا وجد، وإدخال إذا لم يوجد) يتكرر كثيراً في أنظمة الإدارة، مثل ضبط إعدادات المستخدم، أو تحديد أهداف شهرية.

4. الكود الكامل لصفحة budget.php

```
<?php

include 'includes/auth.php';
include 'includes/config.php';

$user_id = (int) $_SESSION['user_id'];
$message = "";

$currentMonth = date('n');
$currentYear = date('Y');

// جلب ميزانية الشهر الحالي إن وجدت
$stmt = $conn->prepare("SELECT budget_amount FROM budgets WHERE
user_id = ? AND month = ? AND year = ?");
$stmt->bind_param("iii", $user_id, $currentMonth, $currentYear);
$stmt->execute();
$result = $stmt->get_result();
$budget = $result->fetch_assoc();
$stmt->close();

$currentBudget = $budget['budget_amount'] ?? "";

if (isset($_POST['save_budget'])) {
    $month = (int) $_POST['month'];
    $year = (int) $_POST['year'];
    $budget_amount = (float) $_POST['budget_amount'];
```

```

if ($month >= 1 && $month <= 12 && $year >= 2024 &&
$budget_amount > 0) {

    // التحقق إذا كانت الميزانية موجودة مسبقاً

    $checkStmt = $conn->prepare("SELECT id FROM budgets WHERE
user_id = ? AND month = ? AND year = ?");

    $checkStmt->bind_param("iii", $user_id, $month, $year);

    $checkStmt->execute();

    $checkResult = $checkStmt->get_result();

    $existing = $checkResult->fetch_assoc();

    $checkStmt->close();

    if ($existing) {

        $updateStmt = $conn->prepare("UPDATE budgets SET
budget_amount = ? WHERE user_id = ? AND month = ? AND year = ?");

        $updateStmt->bind_param("diii", $budget_amount, $user_id,
$month, $year);

        if ($updateStmt->execute()) {

            $message = "تم تحديث الميزانية الشهرية بنجاح.";

        } else {

            $message = "حدث خطأ أثناء تحديث الميزانية.";

        }

        $updateStmt->close();

    } else {

        $insertStmt = $conn->prepare("INSERT INTO budgets (user_id,
month, year, budget_amount) VALUES (?, ?, ?, ?)");

```



```
$insertStmt->bind_param("iiid", $user_id, $month, $year,
$budget_amount);
```

```
if ($insertStmt->execute()) {
    $message = "تم حفظ الميزانية الشهرية بنجاح.";
} else {
    $message = "حدث خطأ أثناء حفظ الميزانية.";
}
```

```
$insertStmt->close();
}
```

```
$currentBudget = $budget_amount;
$currentMonth = $month;
$currentYear = $year;
} else {
    $message = "يرجى إدخال بيانات صحيحة.";
}
}
?>
```

```
<?php include 'includes/header.php'; ?>
```

```
<div class="container" style="margin-top:30px;">
    <div class="card" style="max-width:600px; margin:auto;">
        <h2>إدخال الميزانية الشهرية</h2>
```

```

<?php if (!empty($message)) : ?>

    <p style="color:green;"><?php echo htmlspecialchars($message,
    ENT_QUOTES, 'UTF-8'); ?></p>

<?php endif; ?>

<form method="post">

    <label>الشهر</label>

    <select name="month" required>

        <?php for ($m = 1; $m <= 12; $m++) : ?>

            <option value="<?php echo $m; ?>" <?php if ($m ==
            $currentMonth) echo 'selected'; ?>>

                <?php echo $m; ?>

            </option>

        <?php endfor; ?>

    </select>

    <br><br>

    <label>السنة</label>

    <input type="number" name="year" value="<?php echo
    $currentYear; ?>" required>

    <br><br>

    <label>الميزانية الشهرية</label>

    <input type="number" name="budget_amount" step="0.01"

```

```

        value="<?php echo htmlspecialchars($currentBudget,
ENT_QUOTES, 'UTF-8'); ?>" required>

        <br><br>

        <button type="submit" name="save_budget" class="btn btn-
primary">حفظ الميزانية</button>

    </form>
</div>
</div>

<?php include 'includes/footer.php'; ?>

```

5. شرح الكود بإيجاز

١. حماية الصفحة وتجهيز المتغيرات

- نستدعي ملفات الحماية والاتصال، ثم نقرأ `user_id` من الجلسة، ونعرف المتغير `$message` لعرض رسائل النجاح أو الخطأ.
- نحدد الشهر والسنة الحاليين باستخدام `date('Y')` و `date('n')`.

٢. جلب ميزانية الشهر الحالي

- نستعلم من جدول `budgets` عن سجل يطابق `user_id + month + year` :
- إذا وُجد، نخزن `budget_amount` في `$currentBudget` ليظهر في الحقل عند فتح الصفحة.

٣. معالجة نموذج الميزانية عند الإرسال

- نقرأ الشهر والسنة والمبلغ من `$_POST`.
- نتحقق من أن الشهر بين ١ و ١٢، والسنة ≤ ٢٠٢٤ ، والمبلغ أكبر من صفر.
- إذا كانت البيانات صحيحة:
- نستعلم أولاً: هل يوجد سجل لهذا المستخدم وهذا الشهر وهذه السنة؟

- إذا وُجد → نستخدم UPDATE لتحديث قيمة budget_amount.
 - إذا لم يوجد → نستخدم INSERT لإضافة سجل جديد في جدول budgets.
 - نحديث المتغيرات \$currentBudget, \$currentMonth, \$currentYear
لنعرض القيم الجديدة في الفورم بعد الحفظ.
٤. نموذج إدخال الميزانية في الواجهة
- حقل اختيار شهر (١-١٢) مع وضع الشهر الحالي كـ selected.
 - حقل السنة بقيمة السنة الحالية.
 - حقل الميزانية الشهرية مع عرض القيمة الحالية إن وجدت.
 - زر "حفظ الميزانية" يرسل النموذج ويعيدنا لنفس الصفحة مع رسالة مناسبة.

الدرس الثامن عشر: لوحة التحكم الرئيسية

مقدمة الدرس

بعد أن أصبح لدينا الآن نظام كامل لإدارة التصنيفات والمصروفات، ننتقل إلى صفحة مهمة جدًا في أي مشروع من هذا النوع وهي **لوحة التحكم الرئيسية**.
لوحة التحكم هي الصفحة الأولى التي يرى فيها المستخدم أهم المعلومات بسرعة دون الحاجة إلى فتح كل الصفحات واحدة واحدة.
فيها يمكن عرض الملخص العام للمصروفات، والميزانية، والتنبيهات، وبعض الرسوم البيانية أو المؤشرات المختصرة.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- ما هي لوحة التحكم ولماذا نحتاجها.
- كيف ننشئ صفحة رئيسية تعرض أهم البيانات.
- كيف نجلب الإحصائيات من قاعدة البيانات.
- كيف نعرض البيانات بشكل مرتب وواضح.
- كيف نجعل الصفحة مناسبة للمبتدئين وسهلة الاستخدام.

ما هي لوحة التحكم؟

لوحة التحكم هي واجهة مختصرة تعرض للمستخدم معلومات مهمة جدًا في مكان واحد.
بدلاً من أن يدخل إلى صفحة المصروفات والتقارير والميزانية والتصنيفات كل مرة، يكفي أن يفتح لوحة التحكم ليعرف:

- كم صرف هذا الشهر.
- كم بقي من الميزانية.
- هل هناك تنبيه مهم.
- ما أكثر تصنيف تم استخدامه.
- ما وضع حسابه بشكل عام.

لماذا هي مهمة؟

لوحة التحكم مهمة جدًا لأنها تعطي المستخدم صورة سريعة عن حالته المالية.
وفي المشاريع العملية تُعتبر هذه الصفحة من أهم الصفحات لأنها تجمع البيانات من أكثر من جدول في مكان واحد.

اسم الصفحة

عادة نسميها:

dashboard.php

ماذا تعرض الصفحة؟

يمكن أن تعرض اللوحة:

- إجمالي المصروفات لهذا الشهر.
- إجمالي الميزانية.
- نسبة الصرف.
- المبلغ المتبقي.
- عدد المصروفات المسجلة.
- أكثر تصنيف تم الصرف عليه.
- التنبيهات المهمة.
- بعض الرسوم البيانية البسيطة.

كود لوحة التحكم

```
<?php
include 'includes/auth.php';
include 'includes/config.php';
include 'includes/header.php';

$user_id = (int) $_SESSION['user_id'];

// الشهر والسنة المختاران من المستخدم
$selectedMonth = isset($_GET['month']) ? (int)$_GET['month'] :
(int)date('m');
$selectedYear = isset($_GET['year']) ? (int)$_GET['year'] : (int)date('Y');
```

```

الشهر والسنة السابقان للمقارنة //
$previousMonth = $selectedMonth - 1;
$previousYear = $selectedYear;

if ($previousMonth < 1) {
    $previousMonth = 12;
    $previousYear = $selectedYear - 1;
}

إجمالي مصروفات الشهر المختار //
$expenseStmt = $conn->prepare("
    SELECT COALESCE(SUM(amount), 0) AS total_expenses
    FROM expenses
    WHERE user_id = ? AND MONTH(expense_date) = ? AND
YEAR(expense_date) = ?
");

$expenseStmt->bind_param("iii", $user_id, $selectedMonth,
$selectedYear);

$expenseStmt->execute();

$expenseResult = $expenseStmt->get_result();
$expenseData = $expenseResult->fetch_assoc();
$expenseStmt->close();

$totalExpenses = (float)($expenseData['total_expenses'] ?? 0);

الميزانية للشهر المختار //
$budgetStmt = $conn->prepare("

```

```

SELECT budget_amount
FROM budgets
WHERE user_id = ? AND month = ? AND year = ?
");
$budgetStmt->bind_param("iii", $user_id, $selectedMonth,
$selectedYear);
$budgetStmt->execute();
$budgetResult = $budgetStmt->get_result();
$budgetData = $budgetResult->fetch_assoc();
$budgetStmt->close();

$monthlyBudget = (float)($budgetData['budget_amount'] ?? 0);

// المتبقي والنسبة
$remaining = $monthlyBudget - $totalExpenses;
if ($remaining < 0) {
    $remaining = 0;
}

$percentage = 0;
if ($monthlyBudget > 0) {
    $percentage = ($totalExpenses / $monthlyBudget) * 100;
}

// مصروفات الشهر السابق للمقارنة
$prevStmt = $conn->prepare("
SELECT COALESCE(SUM(amount), 0) AS prev_expenses

```



```

FROM expenses

WHERE user_id = ? AND MONTH(expense_date) = ? AND
YEAR(expense_date) = ?

");

$prevStmt->bind_param("iii", $user_id, $previousMonth,
$previousYear);

$prevStmt->execute();

$prevResult = $prevStmt->get_result();

$prevData = $prevResult->fetch_assoc();

$prevStmt->close();

$previousExpenses = (float)($prevData['prev_expenses'] ?? 0);

// أكثر التصنيفات استخدامًا في الشهر المختار
$categoryStmt = $conn->prepare("

SELECT categories.name, SUM(expenses.amount) AS total_amount

FROM expenses

INNER JOIN categories ON expenses.category_id = categories.id

WHERE expenses.user_id = ? AND MONTH(expenses.expense_date) =
? AND YEAR(expenses.expense_date) = ?

GROUP BY categories.name

ORDER BY total_amount DESC

");

$categoryStmt->bind_param("iii", $user_id, $selectedMonth,
$selectedYear);

$categoryStmt->execute();

$categoryResult = $categoryStmt->get_result();

```

```

$chartLabels = [];
$chartData = [];
$topCategories = [];

while ($row = $categoryResult->fetch_assoc()) {
    $topCategories[] = $row;
    $chartLabels[] = $row['name'];
    $chartData[] = (float)$row['total_amount'];
}
$categoryStmt->close();

// حالة الميزانية
$percentageUsed = 0;
$budgetStatus = "لم يتم تحديد الميزانية";
$budgetClass = "card-teal";
$alertMessage = "";
$alertClass = "";

if ($monthlyBudget > 0) {
    $percentageUsed = ($totalExpenses / $monthlyBudget) * 100;

    if ($percentageUsed >= 100) {
        $budgetStatus = "تم تجاوز الميزانية";
        $budgetClass = "card-red";
        $alertMessage = ".تنبيه: تم تجاوز الميزانية للشهر المحدد";
    }
}

```

```

    $alertClass = "danger";
} elseif ($percentageUsed >= 80) {
    $budgetStatus = "قريب من تجاوز الميزانية";
    $budgetClass = "card-orange";
    $alertMessage = "تنبيه: أنت قريبة من تجاوز الميزانية للشهر المحدد";
    $alertClass = "warning";
} else {
    $budgetStatus = "الميزانية مستقرة";
    $budgetClass = "card-green";
}
}

$monthNames = [
    1 => 'يناير',
    2 => 'فبراير',
    3 => 'مارس',
    4 => 'أبريل',
    5 => 'مايو',
    6 => 'يونيو',
    7 => 'يوليو',
    8 => 'أغسطس',
    9 => 'سبتمبر',
    10 => 'أكتوبر',
    11 => 'نوفمبر',
    12 => 'ديسمبر'
];

```

```
$selectedMonthName = $monthNames[$selectedMonth] ??  
$selectedMonth;  
$printDate = date('Y-m-d H:i');  
?>
```

```
<style>
```

```
.cards-row{  
    display:flex;  
    flex-wrap:wrap;  
    gap:20px;  
    margin-top:30px;  
}
```

```
.cards-row .card{  
    flex:1 1 260px;  
    width:auto !important;  
}
```

```
.print-btn-box{  
    margin-bottom:20px;  
    text-align:left;  
}
```

```
.print-btn{  
    display:inline-block;  
    background:#111827;
```

```
color:#fff;
padding:10px 18px;
border-radius:10px;
text-decoration:none;
font-weight:bold;
transition:0.3s;
}

.print-btn:hover{
    background:#374151;
    color:#fff;
}

.card.card-blue{
    background:linear-gradient(135deg, #2563eb, #3b82f6);
}

.card.card-green{
    background:linear-gradient(135deg, #10b981, #34d399);
}

.card.card-orange{
    background:linear-gradient(135deg, #f59e0b, #fbbf24);
    color:#111827;
}
```

```
.card.card-red{  
    background:linear-gradient(135deg, #ef4444, #f87171);  
}
```

```
.card.card-purple{  
    background:linear-gradient(135deg, #7c3aed, #a78bfa);  
}
```

```
.card.card-teal{  
    background:linear-gradient(135deg, #14b8a6, #2dd4bf);  
}
```

```
.card.card-sky{  
    background:linear-gradient(135deg, #0ea5e9, #38bdf8);  
}
```

```
.card.card-pink{  
    background:linear-gradient(135deg, #ec4899, #f472b6);  
}
```

```
.card.card-gray{  
    background:linear-gradient(135deg, #6b7280, #9ca3af);  
    color:#f9fafb;  
}
```

```
.card.card-offwhite{
```

```
background:linear-gradient(135deg, #f9fafb, #e5e7eb);
color:#111827;
border:1px solid #e5e7eb;
}

.report-print-header,
.report-print-footer{
    display:none;
}

@media print {
    @page{
        margin: 20mm 12mm;
    }

    body * {
        visibility: hidden;
    }

    #print-area,
    #print-area * {
        visibility: visible;
    }

    #print-area {
        position: absolute;
```

```
top: 0;
left: 0;
width: 100%;
background: #fff;
padding-top: 120px;
padding-bottom: 80px;
}
```

```
.report-print-header,
.report-print-footer{
    display: block;
    position: fixed;
    left: 0;
    width: 100%;
    background: #fff;
    color: #111;
    z-index: 9999;
}
```

```
.report-print-header{
    top: 0;
    padding: 10px 20px;
    border-bottom: 2px solid #d1d5db;
}
```

```
.report-print-footer{
```



```
    bottom: 0;
    padding: 10px 20px;
    border-top: 1px solid #d1d5db;
    font-size: 13px;
}
```

```
.print-header-title{
    font-size: 22px;
    font-weight: bold;
    margin-bottom: 5px;
}
```

```
.print-header-subtitle{
    font-size: 14px;
    color: #374151;
}
```

```
.print-footer-content{
    display: flex;
    justify-content: space-between;
    align-items: center;
}
```

```
.no-print{
    display: none !important;
}
```

```
.cards-row{
  display:block;
}

.cards-row .card{
  width:100% !important;
  margin-bottom:15px;
  page-break-inside: avoid;
  break-inside: avoid;
}

.card{
  color:#111 !important;
  box-shadow:none !important;
}

canvas{
  max-width:100% !important;
}
}
</style>

<div class="container" style="margin-top:30px;">
  <div class="card">
```

```

<div class="print-btn-box no-print">
    <a href="#" onclick="printReport(); return false;" class="print-
btn">طباعة التقرير</a>
</div>

<h2>لوحة التحكم</h2>
<p>من هذه الصفحة يمكنك اختيار شهر وسنة محددتين، ثم مشاهدة ملخص مالي كامل
مع مقارنة ورسوم بيانية.</p>

<div class="card no-print" style="max-width:600px; margin:auto;">
    <form method="get" style="margin-top:20px;">
        <label>الشهر</label>
        <input type="number" name="month" min="1" max="12"
            value="<?php echo htmlspecialchars($selectedMonth,
ENT_QUOTES, 'UTF-8'); ?>" required>

        <br><br>

        <label>السنة</label>
        <input type="number" name="year"
            value="<?php echo htmlspecialchars($selectedYear,
ENT_QUOTES, 'UTF-8'); ?>" required>

        <br><br>

        <button type="submit" class="btn btn-primary">عرض
الملخص</button>
    </form>

```

```

</div>

<?php if (!empty($alertMessage)) : ?>
    <div class="alert <?php echo $alertClass; ?> no-print"
style="margin-top:20px;">
        <?php echo htmlspecialchars($alertMessage, ENT_QUOTES,
'UTF-8'); ?>
    </div>
<?php endif; ?>

<div id="print-area">

    <div class="report-print-header">
        <div class="print-header-title">Expense Tracker - التقرير
الشهري</div>
        <div class="print-header-subtitle">
            <?php echo htmlspecialchars($selectedMonthName,
ENT_QUOTES, 'UTF-8'); ?>
            <?php echo htmlspecialchars($selectedYear, ENT_QUOTES,
'UTF-8'); ?>
        </div>
    </div>

    <div class="report-print-footer">
        <div class="print-footer-content">
            <span>تاريخ الطباعة: <?php echo htmlspecialchars($printDate,
ENT_QUOTES, 'UTF-8'); ?></span>
            <span>Expense Tracker</span>
        </div>
    </div>
</div>

```

```

    </div>

</div>

<div class="card card-colored <?php echo $budgetClass; ?>"
style="margin-top:20px;">
    <h3>حالة الميزانية</h3>
    <p><?php echo htmlspecialchars($budgetStatus, ENT_QUOTES,
'UTF-8'); ?></p>
    <?php if ($monthlyBudget > 0): ?>
        <small><?php echo
number_format($percentageUsed, 1); ?>% من الميزانية تم استهلاك</small>
    <?php else: ?>
        <small>لم يتم إدخال ميزانية لهذا الشهر</small>
    <?php endif; ?>
</div>

<div class="cards-row">

    <div class="card card-colored card-blue">
        <h3>إجمالي المصروفات</h3>
        <p><?php echo number_format($totalExpenses, 2); ?>
ريال</p>
    </div>

    <div class="card card-colored card-gray">
        <h3>الميزانية الشهرية</h3>
        <p><?php echo number_format($monthlyBudget, 2); ?>
ريال</p>

```

</div>

<div class="card card-colored card-teal">

<h3>المبلغ المتبقي</h3>

<p><?php echo number_format(\$remaining, 2); ?> ريال</p>

</div>

<div class="card card-colored card-purple">

<h3>نسبة الصرف</h3>

<p><?php echo round(\$percentage, 2); ?>%</p>

</div>

<div class="card card-colored card-sky">

<h3>مقارنة بين الشهر الحالي والسابق</h3>

<p>مصروفات الشهر الحالي <?php echo number_format(\$totalExpenses, 2); ?> ريال</p>

<p>مصروفات الشهر السابق <?php echo number_format(\$previousExpenses, 2); ?> ريال</p>

</div>

<div class="card card-colored card-pink">

<h3>أكثر التصنيفات استخدامًا</h3>

<?php if (count(\$topCategories) === 0): ?>

<p>لا توجد بيانات مصروفات لهذا الشهر</p>

<?php else: ?>

<?php foreach (\$topCategories as \$category): ?>

```

        </li>

        <?php echo htmlspecialchars($category['name'],
ENT_QUOTES, 'UTF-8'); ?>

        - <?php echo
number_format($category['total_amount'], 2); ?> ريال

    </li>

    <?php endforeach; ?>

</ul>

<?php endif; ?>

</div>

</div>

<div class="card" style="margin-top:30px;">
    <h3>الرسوم البيانية</h3>

    <canvas id="summaryChart" style="max-width:100%;
height:400px;"></canvas>

</div>

</div>

</div>

</div>

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<script>
const ctx = document.getElementById('summaryChart').getContext('2d');

```

```

new Chart(ctx, {
    type: 'bar',
    data: {
        labels: <?php echo json_encode($chartLabels,
JSON_UNESCAPED_UNICODE); ?>,
        datasets: [{
            label: 'إجمالي المصروفات',
            data: <?php echo json_encode($chartData,
JSON_NUMERIC_CHECK); ?>,
            backgroundColor: ['#3498db', '#2ecc71', '#f39c12', '#e74c3c',
'#9b59b6', '#1abc9c', '#8e44ad']
        }]
    },
    options: {
        responsive: true,
        scales: {
            y: {
                beginAtZero: true
            }
        }
    }
});

function printReport() {
    window.print();
}
</script>

```



```
<?php include 'includes/footer.php'; ?>
```

فكرة الكود:

هذه الصفحة تجمع ملخصاً شهرياً للمصروفات من قاعدة البيانات، وتعرضه في كروت ورسوم بيانية، مع إمكانية اختيار شهر/سنة وطباعة تقرير منسق.

الفكرة في ٤ نقاط

١. قراءة المدخلات والحسابات

الكود يقرأ `user_id` من الجلسة، ثم يأخذ الشهر والسنة من النموذج (أو من التاريخ الحالي)، ويحسب: إجمالي مصروفات الشهر، الميزانية، المبلغ المتبقي، نسبة الصرف، مصروفات الشهر السابق، ومجموع المصروفات لكل تصنيف باستخدام استعلامات SUM و GROUP BY

٢. تحديد حالة الميزانية

بناءً على نسبة استخدام الميزانية يتم تحديد حالة الميزانية (مستقرة – قريب من التجاوز – متجاوزة) مع تغيير لون الكرت وإظهار رسالة تنبيه مناسبة للمستخدم.

٣. عرض الملخص والتقرير

في الـ HTML تظهر:

- نموذج لاختيار الشهر والسنة.
- كروت تلخص: إجمالي المصروفات، الميزانية، المتبقي، نسبة الصرف، مقارنة مع الشهر السابق، وأشهر التصنيفات.
- كرت للرسم البياني (مخطط أعمدة) يوضح المصروفات حسب التصنيف باستخدام مكتبة Chart.js

٤. ميزة طباعة التقرير

يوجد زر "طباعة التقرير" يستدعي `window.print()`، ومع تنسيقات `@media print` يتم إخفاء بقية الصفحة وإظهار جزء التقرير فقط (`#print-area`) مع ترويسة وفوتر خاصين بالتقرير يحتويان اسم النظام، شهر التقرير، وتاريخ الطباعة. بهذا الشكل تستطيع الطالبة أن تفهم أن الصفحة هي **Dashboard** تقرير شهري قابل للطباعة مبني على استعلامات MySQL ورسم بيانات بـ Chart.js

الدرس التاسع عشر: تعديل الملف الشخصي

مقدمة الدرس

بعد أن أنشأنا نظام تسجيل الدخول والتسجيل، نصل الآن إلى خطوة مهمة جدًا وهي **تعديل الملف الشخصي**. هذه الصفحة تسمح للمستخدم بتحديث بياناته الشخصية مثل الاسم أو البريد الإلكتروني، حتى تبقى معلوماته صحيحة ومحدثة داخل النظام.

هدف الدرس

هدف هذا الدرس هو أن نتعلم:

- كيف نعرض بيانات المستخدم الحالية داخل نموذج.
- كيف نسمح له بتعديل الاسم أو البريد الإلكتروني.
- كيف نحفظ التحديث في قاعدة البيانات.
- كيف نحافظ على الجلسة بعد التعديل.
- كيف نجعل الصفحة سهلة وآمنة للمستخدم.

عادة نسمي هذا الملف:

• profile.php

لماذا نحتاج تعديل الملف الشخصي؟

ميزة تعديل الملف الشخصي مهمة جدًا لأن المستخدم قد:

- يغيّر اسمه.
- يغيّر بريده الإلكتروني.
- يريد تصحيح بيانات أدخلها سابقًا.
- يحتاج إلى الحفاظ على حسابه محدثًا ودقيقًا.

فكرة الصفحة

عندما يفتح المستخدم صفحة الملف الشخصي:

١. نقرأ بياناته من الجلسة أو قاعدة البيانات.
٢. نعرض الاسم والبريد الحاليين داخل النموذج.
٣. يكتب التعديلات المطلوبة.
٤. يضغط على حفظ.

٥. يتم تحديث البيانات في الجدول.

٦. تبقى الجلسة كما هي، أو تُحدَّث القيم الجديدة داخلها.

الخطوات العملية للدرس

الخطوة الأولى: التأكد من تسجيل الدخول

صفحة الملف الشخصي يجب أن تكون محمية، لذلك لا بد من التأكد أن المستخدم مسجل دخوله.

الخطوة الثانية: جلب بيانات المستخدم

سنستخدم `user_id` من الجلسة لجلب بياناته من جدول `users`.

الخطوة الثالثة: عرض النموذج

نعرض:

- الاسم الكامل.
- البريد الإلكتروني.
- زر حفظ التعديلات.

الخطوة الرابعة: حفظ التعديلات

عند الضغط على زر الحفظ، نُحدَّث بيانات المستخدم في قاعدة البيانات.

الخطوة الخامسة: تحديث الجلسة

بعد نجاح التحديث، يمكن تحديث اسم المستخدم داخل الجلسة حتى ينعكس التغيير مباشرة.

كود صفحة تعديل الملف الشخصي

```
<?php
include 'includes/auth.php';
include 'includes/config.php';
include 'includes/header.php';

$user_id = (int) $_SESSION['user_id'];
$message = "";
$messageType = "";
```

```

// جلب بيانات المستخدم الحالية
$stmt = $conn->prepare("SELECT id, full_name, email, password FROM
users WHERE id = ?");
if (!$stmt) {
    die("خطأ في استعمال جلب بيانات المستخدم: " . $conn->error);
}

$stmt->bind_param("i", $user_id);
$stmt->execute();
$result = $stmt->get_result();
$user = $result->fetch_assoc();
$stmt->close();

if (!$user) {
    die("لم يتم العثور على بيانات المستخدم الحالي");
}

// عند حفظ التعديلات
if (isset($_POST['save'])) {
    $full_name = trim($_POST['full_name'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $current_password = $_POST['current_password'] ?? '';
    $new_password = $_POST['new_password'] ?? '';
    $confirm_password = $_POST['confirm_password'] ?? '';

    if (empty($full_name) || empty($email)) {

```

```

$message = "يرجى تعبئة الاسم والبريد الإلكتروني";
$messageType = "error";
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $message = "يرجى إدخال بريد إلكتروني صحيح";
    $messageType = "error";
} else {
    // التحقق من عدم تكرار البريد لمستخدم آخر

    $checkStmt = $conn->prepare("SELECT id FROM users WHERE email
= ? AND id != ?");

    if (!$checkStmt) {
        die("$conn->error: خطأ في استعلام التحقق من البريد الإلكتروني");
    }

    $checkStmt->bind_param("si", $email, $user_id);
    $checkStmt->execute();
    $checkResult = $checkStmt->get_result();

    if ($checkResult->num_rows > 0) {
        $message = "هذا البريد الإلكتروني مستخدم بالفعل";
        $messageType = "error";
    } else {
        // إذا كان المستخدم يريد تغيير كلمة المرور

        if (!empty($current_password) || !empty($new_password) ||
!empty($confirm_password)) {

            if (empty($current_password) || empty($new_password) ||
empty($confirm_password)) {

```

```

$message = "لتغيير كلمة المرور يجب تعبئة جميع حقول كلمة المرور";
$messageType = "error";
} elseif (!password_verify($current_password,
$user['password'])) {
    $message = "كلمة المرور الحالية غير صحيحة.";
    $messageType = "error";
} elseif (strlen($new_password) < 6) {
    $message = "يجب أن تكون كلمة المرور الجديدة ٦ أحرف على الأقل";
    $messageType = "error";
} elseif ($new_password !== $confirm_password) {
    $message = "تأكيد كلمة المرور غير مطابق.";
    $messageType = "error";
} else {
    $hashedPassword = password_hash($new_password,
PASSWORD_DEFAULT);

    $updateStmt = $conn->prepare("UPDATE users SET full_name
= ?, email = ?, password = ? WHERE id = ?");
    if (!$updateStmt) {
        die("$conn->error: خطأ في استعلام تحديث كلمة المرور");
    }

    $updateStmt->bind_param("sssi", $full_name, $email,
$hashedPassword, $user_id);

    if ($updateStmt->execute()) {
        $message = "تم تحديث الملف الشخصي وكلمة المرور بنجاح";
    }
}

```

```

$messageType = "success";

$user['full_name'] = $full_name;
$user['email'] = $email;
$user['password'] = $hashedPassword;
} else {
    $message = "حدث خطأ أثناء تحديث البيانات";
    $messageType = "error";
}

$updateStmt->close();
}

} else {
    // تحديث الاسم والبريد فقط
    $updateStmt = $conn->prepare("UPDATE users SET full_name =
    ?, email = ? WHERE id = ?");
    if (!$updateStmt) {
        die("خطأ في استعمال تحديث البيانات");
    }

    $updateStmt->bind_param("ssi", $full_name, $email, $user_id);

    if ($updateStmt->execute()) {
        $message = "تم تحديث الملف الشخصي بنجاح";
        $messageType = "success";
    }
}

```

```

        $user['full_name'] = $full_name;
        $user['email'] = $email;
    } else {
        $message = "حدث خطأ أثناء تحديث البيانات" . $updateStmt->error;
        $messageType = "error";
    }

    $updateStmt->close();
}
}

$checkStmt->close();
}
}
?>

<style>
.profile-wrapper{
    max-width: 850px;
    margin: 30px auto;
}

.profile-card{
    padding: 25px;
}

```



```
.profile-grid{  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 20px;  
}
```

```
.profile-grid-full{  
  grid-column: 1 / -1;  
}
```

```
.profile-info-box{  
  background: #f8fafc;  
  border: 1px solid #e5e7eb;  
  border-radius: 12px;  
  padding: 15px;  
  margin-bottom: 20px;  
}
```

```
.profile-info-box p{  
  margin: 8px 0;  
}
```

```
.alert-success{  
  background: #dcfce7;  
  color: #166534;
```

```
padding: 12px 15px;
border-radius: 10px;
margin-bottom: 20px;
}

.alert-error{
background: #fee2e2;
color: #991b1b;
padding: 12px 15px;
border-radius: 10px;
margin-bottom: 20px;
}

.section-title{
margin-top: 25px;
margin-bottom: 10px;
font-size: 18px;
}

@media (max-width: 768px){
.profile-grid{
grid-template-columns: 1fr;
}
}

</style>
```

```

<div class="container profile-wrapper">
  <div class="card profile-card">
    <h2>الملف الشخصي</h2>
    <p>من هذه الصفحة يمكنك تعديل بيانات حسابك وتغيير كلمة المرور</p>

    <div class="profile-info-box">
      <p><strong>رقم المستخدم</strong> <?php echo (int)$user['id'];
?></p>
      <p><strong>الاسم الحالي</strong> <?php echo
htmlspecialchars($user['full_name'], ENT_QUOTES, 'UTF-8'); ?></p>
      <p><strong>البريد الإلكتروني الحالي</strong> <?php echo
htmlspecialchars($user['email'], ENT_QUOTES, 'UTF-8'); ?></p>
    </div>

    <?php if (!empty($message)) : ?>
      <div class="alert-<?php echo $messageType === 'success' ?
'success' : 'error'; ?>">
        <?php echo htmlspecialchars($message, ENT_QUOTES, 'UTF-8');
?>
      </div>
    <?php endif; ?>

    <form method="post">
      <div class="profile-grid">
        <div>
          <label>الاسم</label>
          <input type="text" name="full_name"

```

```
value="<?php echo htmlspecialchars($user['full_name'],
ENT_QUOTES, 'UTF-8'); ?>" required>
```

```
</div>
```

```
<div>
```

```
<label>البريد الإلكتروني</label>
```

```
<input type="email" name="email"
```

```
value="<?php echo htmlspecialchars($user['email'],
ENT_QUOTES, 'UTF-8'); ?>" required>
```

```
</div>
```

```
</div>
```

```
<h3 class="section-title">تغيير كلمة المرور</h3>
```

```
<p>اترك الحقول التالية فارغة إذا كنت لا تريد تغيير كلمة المرور</p>
```

```
<div class="profile-grid">
```

```
<div class="profile-grid-full">
```

```
<label>كلمة المرور الحالية</label>
```

```
<input type="password" name="current_password">
```

```
</div>
```

```
<div>
```

```
<label>كلمة المرور الجديدة</label>
```

```
<input type="password" name="new_password">
```

```
</div>
```

```
<div>
```

```

<label>تأكيد كلمة المرور الجديدة</label>

<input type="password" name="confirm_password">

</div>

</div>

<br>

<button type="submit" name="save" class="btn btn-
primary">حفظ التعديلات</button>

<a href="dashboard.php" class="btn btn-secondary">العودة إلى لوحة
التحكم</a>

</form>

</div>

</div>

<?php include 'includes/footer.php'; ?>

```

شرح الكود

1) بدء الجلسة session_start();

نحتاج الجلسة للوصول إلى بيانات المستخدم الحالي.

2) التأكد من تسجيل الدخول

```

if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

```

هذا يمنع فتح الصفحة بدون دخول.

3) جلب بيانات المستخدم

SELECT full_name, email FROM users WHERE id = ?

هذا الاستعلام يجلب الاسم والبريد الخاص بالمستخدم الحالي.

4) عرض القيم داخل النموذج

```
value="<?php echo htmlspecialchars($user['full_name']); ?>"
```

هكذا يظهر الاسم الحالي داخل الحقل، ويمكن للمستخدم تعديله مباشرة.

5) التحقق من البريد الإلكتروني المكرر

```
SELECT id FROM users WHERE email = ? AND id != ?
```

هذا يمنع استخدام بريد إلكتروني موجود لحساب آخر.

6) تحديث البيانات

```
UPDATE users SET full_name = ?, email = ? WHERE id = ?
```

هذا السطر يحفظ التعديلات في قاعدة البيانات.

7) تحديث الجلسة

```
$_SESSION['user_name'] = $full_name;
```

```
$_SESSION['user_email'] = $email;
```

حتى يظهر الاسم الجديد مباشرة في الموقع.

لماذا هذا الدرس مهم؟

لأن الملف الشخصي جزء أساسي من أي نظام مستخدمين. فمن الطبيعي أن يرغب المستخدم في تحديث بياناته الشخصية دون إنشاء حساب جديد.

ملاحظات مهمة للمبتدئين

- لا تنسي حماية الصفحة بتسجيل الدخول.
- افحصي البريد الإلكتروني المكرر قبل التحديث.
- حدّثي الجلسة بعد النجاح حتى تظهر القيم الجديدة.
- استخدمي htmlspecialchars عند عرض القيم داخل النموذج.
- يمكن لاحقًا إضافة صورة شخصية أو تغيير كلمة المرور من نفس الصفحة.

مثال عملي

إذا كان اسم المستخدم قد تغيّر من:

- سارة أحمد

إلى:

• سارة محمد

فيمكنها فتح صفحة الملف الشخصي، تعديل الاسم، ثم حفظ التعديلات، وسيتم تحديثه في النظام مباشرة.

خلاصة الدرس

في هذا الدرس تعلمنا كيف نُنشئ صفحة تعديل الملف الشخصي. تعرفنا على كيفية جلب بيانات المستخدم الحالي، وعرضها داخل النموذج، والتحقق من تكرار البريد، ثم حفظ التحديث في قاعدة البيانات وتحديث الجلسة بعد ذلك. وبهذا أصبح المستخدم قادرًا على إدارة بياناته الشخصية داخل النظام بسهولة وأمان.